

AA-mDLAM: An accelerated ADMM-based framework for training deep neural networks

Zeinab Ebrahimi^a, Gustavo Batista^b, Mohammad Deghat^a

^a School of Mechanical and Manufacturing Engineering, University of New South Wales, Sydney, NSW 2052, Australia

^b School of Computer Science and Engineering, University of New South Wales, Sydney, NSW 2052, Australia

ARTICLE INFO

Communicated by L. Tong

Keywords:

Deep learning
Alternating direction method of multipliers
Anderson acceleration
Convergence

ABSTRACT

Stochastic gradient descent (SGD) and its many variants are the widespread optimization algorithms for training deep neural networks. However, SGD suffers from inevitable drawbacks, including vanishing gradients, lack of theoretical guarantees, and substantial sensitivity to input. The Alternating Direction Method of Multipliers (ADMM) has been proposed to address these shortcomings as an effective alternative to the gradient-based methods. It has been successfully employed for training deep neural networks. However, ADMM-based optimizers have a slow convergence rate. This paper proposes an accelerated framework for training deep neural networks, termed AA-mDLAM, which integrates Anderson acceleration within an Alternating Minimization approach inspired by ADMM to tackle this drawback. The main intention of the AA-mDLAM algorithm is to employ Anderson acceleration to alternating minimization by considering it as a fixed-point iteration and attaining a nearly quadratic convergence rate. We verify the effectiveness and efficiency of the proposed AA-mDLAM algorithm by conducting extensive experiments on seven benchmark datasets contrary to other state-of-the-art optimizers.

1. Introduction

Stochastic gradient descent (SGD) and its many variants are well-known methods used as optimization algorithms for training deep neural networks, which apply the back-propagation method. However, multiple downsides of SGD pose a barrier to its outspread applications. For example, it suffers from a gradient vanishing phenomenon in which the error signal decreases as the gradient is backpropagated [1]. In addition, SGD is immensely sensitive to poor conditioning, and therefore a slight change in the input would result in a drastic alteration in the gradient [2]. Moreover, there is a lack of theoretical guarantees for highly nonsmooth and nonconvex deep neural networks. In addition to SGD and its variants, various first-order splitting methods such as the stochastic primal-dual splitting method [3,4] and the stochastic Peaceman–Rachford method [5,6] have been introduced for solving constrained optimization problems. While these methods provide effective splitting strategies, their performance in deep neural network training is often hindered by issues such as high variance in stochastic updates, limited theoretical guarantees in nonconvex settings, and challenges in handling large-scale structured optimization problems.

In recent years, to address these drawbacks, a recognized optimization scheme, namely the Alternating Direction Method of Multipliers (ADMM), has been introduced as an alternative to SGD for resolving

deep learning problems. ADMM aims to divide a problem into several subproblems and then cooperate to achieve a global solution [7]. ADMM has been established as a successful approach to traditional machine learning [8–12]. The following merits make ADMM a promising strategy: it carries out without gradient steps and circumvents gradient vanishing phenomena; it shows great scalability in the matter of the number of layers, training in parallel across the cores; it enables using the nondifferentiable activation functions and nonsmooth regularization term; it is resistant to the poor conditioning problems [8]. Thus, many researchers have explored the ADMM for various deep neural networks. It was primarily applied to resolve the Multi-Layer Perceptron (MLP) problem with convergence guarantees [8,11]. Afterwards, it was expanded to other structures, including Recurrent Neural Networks (RNN) [13], and allowed parallel deep neural network training using multiple GPUs [10,14].

Despite the popularity of applying ADMM to deep neural networks, there exist some significant challenges: (i) Cubic time complexity in respect of feature dimensions. This computational bottleneck stems from the matrix inversion used for updating the weight parameters. Additional sub-iterations are required for obtaining an inverse matrix that leads to the time complexity of $O(n^3)$, in which n is a feature dimension [7]. Taylor et al. [8] inferred from their experiments that

* Corresponding author.

E-mail address: z.ebrahimi@unsw.edu.au (Z. Ebrahimi).

more than 7000 cores are involved in training a neural network with 300 neurons based on ADMM. (ii) Slow convergence towards a high accuracy solution. Even though ADMM is a competent optimizer for large-scale deep learning problems, it takes a long period to converge towards a solution with high precision. The convergence rate of deep learning via ADMM (dLADMM) methods [11] and block coordinate descent (BCD) algorithms in deep learning [12] is sublinear of $O(1/k)$, where k is the number of epochs. Recently, Wang et al. [15] improved the convergence rate by proposing an mDLAM algorithm, which can reach a linear convergence rate. However, to the best of the authors' knowledge, a theoretical framework to further speed up the convergence rate is not developed.

This study proposes a novel optimization structure to tackle these challenges: the Anderson Acceleration for Deep Learning Alternation Direction Method of Multipliers (AA-mDLAM). To update the weight parameters, the quadratic estimation and backtracking algorithms are applied as an alternative to the matrix inversion, resulting in decreasing the time complexity from $O(n^3)$ to $O(n^2)$. Moreover, an acceleration method is used for nonconvex ADMM. The Anderson acceleration technique can accelerate the convergence rate by interpreting ADMM as a fixed-point iteration on an abstracted set of variables. Anderson acceleration exerts a quasi-Newton scheme, which seeks out an almost quadratic convergence rate. We evaluate our proposed AA-mDLAM algorithm on seven benchmark datasets. Thanks to its acceleration approach and theoretical guarantee, our method attains better convergence than other comparison methods, including SGD and its variants and ADMM-based methods, in many experiments. Our key contributions in this paper are recapped as follows:

- We propose a novel optimization framework for deep neural networks. Backtracking and quadratic estimation are applied instead of matrix inversion.
- We present an accelerated algorithm for deep neural networks. The Anderson acceleration approach boosts the convergence rate to nearly quadratic. Anderson Acceleration improves ADMM by leveraging a combination of past iterates to construct a more informed update direction. Specifically, instead of relying only on the most recent iterate, AA utilizes a weighted combination of previous iterates to approximate a fixed-point solution more efficiently. This approach reduces oscillations in weight updates, stabilizes convergence in nonconvex settings, and decreases the total number of iterations required to reach an optimal solution.
- We provide the convergence of our accelerated optimization algorithm subject to proper assumptions, surpassing the existing methods.
- We demonstrate the efficiency and performance of our proposed method by conducting comprehensive experiments on seven benchmark datasets.

The rest of this paper is arranged as follows: In Section 2, we outline the recent research respecting this study. Section 3 describes the problem statement and presents the proposed AA-mDLAM algorithm for training deep neural networks. In Section 4, we provide the details of the convergence properties of the proposed AA-mDLAM algorithm. In Section 5, we present the extensive experimental results to indicate and verify the efficiency of the proposed approach. Finally, Section 6 gives the conclusion of this work.

2. Related work

The literature on deep learning optimization techniques is divided into two main categories: SGD algorithms and alternating minimization schemes, which are outlined as follows:

SGD algorithms: Rumelhart et al. [16] are the first researchers to introduce the well-known backpropagation algorithm to effectually train neural networks. In 2012, deep learning attracted the attention

of many researchers owing to the high accuracy of AlexNet [17]. Consequently, various GD-based optimization algorithms have been presented to accelerate the convergence rate, such as Polyak momentum [18], initialization, and Nesterov momentum [19]. In the past decades, plenty of SGD methods that integrated adaptive learning rates have been proposed, in particular, AdaGrad [20], RMSProp [9], Adam [21], AMSGrad [22], Adabelief [23], and Adabound [24]. The convergence assumption of these adaptive optimizers does not apply to deep learning problems due to the existence of the ReLU as a non-differentiable activation function.

Alternating minimization: Many recent studies have exerted the Alternating Direction Method of Multipliers (ADMM) [7], auxiliary coordinate (MAC) [25], and Block Coordinate Descent (BCD) [12] for constrained neural network problems. In these algorithms, the gradient vanishing problem has not occurred, and non-differentiable activation functions, e.g., ReLU and nonsmooth regularization terms, can be adopted. Li et al. [26] introduced an ADMM-based algorithm to acquire distributed learning for Graph Convolutional Network (GCN). Wen et al. [27] introduced a fast ADMM (fdLADMM) framework that leverages quadratic approximation and backtracking methods to avoid matrix inversion, achieving superior convergence and efficiency across various tasks. Similarly, Xu et al. [28] developed a unified framework to analyze the convergence rates of alternating minimization methods, providing insights into their application to neural network training with enhanced theoretical guarantees. Moreover, Liu et al. [29] proposed the ADMM-LSTM framework, which addresses exploding and vanishing gradient issues while improving computational efficiency through a backward-forward parameter update order. These recent contributions underscore the growing interest and innovations in gradient-free optimization techniques for training deep learning models.

ADMM for nonconvex problem: Despite the leading performance of ADMM for nonconvex problems, its convergence property remains largely open and is not as well-established as for convex problems. The convergence analysis of nonconvex ADMM is challenging due to the lack of Fejér monotonicity in iterations, the complexity of equality and inequality constraints, and coupled objectives [30]. Wang et al. [31] considered mild convergence assumptions for coupled and nonsmooth objective functions of nonconvex ADMM. Chen et al. [32] explored the application of ADMM for quadratic coupling problems. Wang et al. [33] analyzed the convergence of nonconvex ADMM in the presence of constrained equality problems. Furthermore, recent works have extended ADMM for nonconvex and nonsmooth optimization. [34] introduces an upper-bound minimization method for structured updates but relies on proxy functions. [35] proposes an inexact ADMM (I-ADMM) with expansion line search and momentum-based acceleration. Although ADMM has been implemented in various deep learning problems, a lack of theoretical convergence analysis has remained.

Anderson acceleration: Anderson acceleration [36] is an acknowledged approach for increasing the speed of convergence of a fixed-point iteration. It has a wide variety of applications in computational physics [37], numerical linear algebra [38], and robotics [39]. The main conception of Anderson acceleration is using m previous iterations to estimate a new iterate so that it converges more rapidly to the fixed point. It should be noted that this method is a quasi-Newton technique to retrieve the root of the residual function [40]. Recently, Zhang et al. [41] accelerated the convergence of nonconvex ADMM by applying Anderson acceleration in computer graphics.

3. The AA-mDLAM algorithm

In this section, we present our proposed AA-mDLAM algorithm. Table 1 describes the notation utilized in this study. Section 3.1 provides an overview of the existing ADMM-based method for deep learning and then discusses the AA-mDLAM algorithm in detail by reformulating the problem. Section 3.3 investigates the procedure of updating parameters.

Table 1

Notations utilized in this paper.

Notations	Descriptions
L	Number of layers
W_l	The weight matrix in the l th layer
b_l	The intercept vector in the l th layer
z_l	The output of linear mapping for the l th layer
$h_l(z_l)$	The nonlinear activation function for the l th layer
a_l	The output of the l th layer
x	The input matrix of neural networks
y	The predefined label vector
$R(z_l; y)$	The loss function for the l th layer
$\Omega_l(W_l)$	The regularization term for the l th layer
ϵ	The tolerance for the nonlinear mapping
n_l	The number of neurons in the l th layer

3.1. Problem formulation

Generally, MLP includes L layers in which linear mapping and nonlinear activation functions are used. A linear mapping consists of a weight vector $W_l \in \mathbb{R}^{n_l \times n_{l-1}}$, and a nonlinear mapping is determined by a continuous activation function $h_l(\cdot)$. The outputs of the l th layer is $a_l = h_l(W_l a_{l-1})$, where $a_{l-1} \in \mathbb{R}^{n_{l-1}}$ denotes the input of the $(l-1)$ th layer. The neural network problem is mathematically formulated in Problem 1 below by inserting the auxiliary variable z_l as the output of the linear mapping:

Problem 1.

$$\begin{aligned} \min_{W_l, z_l, a_l, b_l} \quad & R(z_L; y) + \sum_{l=1}^L \Omega_l(W_l) \\ \text{s.t.} \quad & z_l = W_l a_{l-1} + b_l \quad (l = 1, \dots, L), \\ & a_l = h_l(z_l) \quad (l = 1, \dots, L-1). \end{aligned} \quad (1)$$

where $a_0 = x \in \mathbb{R}^d$ is the input of the neural network with d as the number of feature dimensions, and y denotes the presumed label vector. The function $R(z_L; y)$ represents the loss function in the L th layer, and $\Omega_l(W_l) \geq 0$ is a regularization term in the l th layer. Both R and Ω_l functions are continuous, convex, and proper¹.

In Problem 1, the nonconvex equality constraint $a_l = h_l(z_l)$ is the most demanding part since the nonlinear activation function is used. Therefore, solving the z_l subproblem and reaching an optimal solution is challenging. Furthermore, there is a lack of guarantee for resolving the nonlinear equality constraints according to alternating minimization optimizers. To overcome the above-mentioned challenges, $h_l(z_l)$ is assumed to be quasilinear [15]. Thus, nonlinear activation functions such as smooth sigmoid, tanh, and the Rectified Linear Unit (ReLU) can be utilized. Accordingly, Problem 1 can be reformulated with inequality constraints by introducing a tolerance ϵ as follows:

Problem 2.

$$\begin{aligned} \min_{W_l, z_l, a_l, b_l} \quad & R(z_L; y) + \sum_{l=1}^L \Omega_l(W_l) \\ \text{s.t.} \quad & z_l = W_l a_{l-1} + b_l \quad (l = 1, \dots, L), \\ & h_l(z_l) - \epsilon \leq a_l \leq h_l(z_l) + \epsilon \quad (l = 1, \dots, L-1). \end{aligned} \quad (2)$$

where $\mathbf{W} = \{W_l\}_{l=1}^L$, $\mathbf{z} = \{z_l\}_{l=1}^L$, $\mathbf{a} = \{a_l\}_{l=1}^L$, $\mathbf{b} = \{b_l\}_{l=1}^L$, and ϵ is exerted to project the nonconvex constraints into ϵ -balls. In Problem 2, the linear constraint $z_l = W_l a_{l-1} + b_l$ can be relaxed with an ℓ_2 penalty term in the objective function, which is formulated as follows:

Problem 3.

$$\begin{aligned} \min_{W_l, z_l, a_l, b_l} \quad & \mathcal{F}(\mathbf{W}, \mathbf{z}, \mathbf{a}, \mathbf{b}) = R(z_L; y) + \sum_{l=1}^L \Omega_l(W_l) \\ & + \frac{\rho}{2} \sum_{l=1}^L \|z_l - W_l a_{l-1} - b_l\|_2^2 \\ \text{s.t.} \quad & h_l(z_l) - \epsilon \leq a_l \leq h_l(z_l) + \epsilon \quad (l = 1, \dots, L-1). \end{aligned} \quad (3)$$

where $\rho > 0$ is a penalty parameter and as $\rho \rightarrow \infty$ and $\epsilon \rightarrow \infty$, Problem 3 approaches Problem 1. Indeed, Problem 3 is still nonconvex due to the existence of $h_l(z_l)$, but it is convex in terms of one variable while others are fixed. To give an example, Problem 3 is convex regarding $\{z_l\}_{l=1}^L$ when $\{W_l\}_{l=1}^L$ and $\{a_l\}_{l=1}^L$ are fixed [15].

There is an implicit issue in solving Problem 3, which relates to having a slow convergence rate. The convergence rate of the dIADMM and mDLAM algorithms to solve Problem 3 is sublinear to $O(1/k)$ and linear, respectively [11,15]. To tackle this issue, we apply the Anderson acceleration methods to elevate the convergence rate.

3.2. Anderson acceleration

The principle of devising a quadratically convergent method that applies solely first-order derivatives is derived from Nesterov's technique to address convex optimization problems. In this method, linear extrapolation of the two previous iterations is assessed for executing iterations instead of the current result exclusively [43]. Even though such a momentum approach can be utilized to speed up the ADMM method, it is improper for nonconvex problems due to the extrapolated point's procedure, which cannot be governed by a bound on the objective function's curve. We summarize the Anderson acceleration below and for more information, the reader can refer to [44].

Consider a fixed-point iteration

$$x_{k+1} = G(x_k) \quad (4)$$

where $x \in \mathbb{R}^n$ and $G : \mathbb{R}^n \rightarrow \mathbb{R}^n$. The purpose of Anderson acceleration is to achieve an almost quadratic convergence rate by deducting the quasi-Newton strategy. For a solution x^* to the fixed-point iteration (4), the residual

$$F(x) = G(x) - x \quad (5)$$

must become zero. The fundamental idea of Anderson acceleration is to utilize the residuals of the recent iterate x_k and its previous m iterates $x_{k-1}, \dots, x_{k-m}$ to extract a new iterate x_{k+1}^{AA} , which diminishes the residual to the furthest extent. Particularly, an affine subspace can be spanned by $x_k, x_{k-1}, \dots, x_{k-m}$, where each point can be defined as

$$x(\alpha) = x_k + \sum_{j=1}^m \alpha_j (x_{k-j} - x_k) \quad (6)$$

and $\alpha = (\alpha_1, \dots, \alpha_m)$ denotes affine coordinates. The new step is obtained by this subspace under the fixed-point mapping G :

$$x_{k+1}^{AA} = G(x_k) - \sum_{j=1}^m \alpha_j^* (G(x_{k-j+1}) - G(x_{k-j})) \quad (7)$$

where the coefficients α_j^* , $j = 1, \dots, m$ are derived by solving a linear least-squares problem:

$$(\alpha_1^*, \dots, \alpha_m^*) = \underset{\alpha_1, \dots, \alpha_m}{\operatorname{argmin}} \left\| F(x_k) - \sum_{j=1}^m \alpha_j \left(F(x_{k-j+1}) - F(x_{k-j}) \right) \right\|_2^2 \quad (8)$$

Here $F(x_k) = G(x_k) - x_k$ represents the residual at step k . We define $\Delta F_k = F_{k+1} - F_k$ and $\Delta G_{k-j} = G(x_{k-j+1}) - G(x_{k-j})$, where $F_k = F(x_k)$. The least-squares problem (8) can be solved as follows concerning the linear dependency of $\{\Delta F_{k-j}\}$ [45]:

$$(D^T D) \alpha = D^T F_k \quad (9)$$

¹ A proper convex function is an extended real-valued convex function with a non-empty domain, that never takes on the value $-\infty$ and also is not identically equal to $+\infty$ [42].

where $D = [\Delta F_{k-1}, \dots, \Delta F_{k-m}]$.

The relationship between the Anderson acceleration and ADMM is that the ADMM algorithm can be noticed as fixed-point iterations $(x_k, y_k) \rightarrow (x_{k+1}, y_{k+1}), k = 0, 1, 2, \dots$. To guarantee the stability of Anderson acceleration, some metrics should be determined to assess the efficiency of an accelerated iteration and a fall-back plan since the nonconvex constraints in the optimization are not met. Goldstein et al. [44] indicated that if the problem is convex, the integrated residual is monotonously reduced via ADMM.

3.2.1. Choice of parameter m

Fang et al. [45] asserted that Anderson acceleration can be assumed as a quasi-Newton technique to obtain the root of the residual function, using the m previous iterates to estimate the inverse Jacobian. Thus, the choice of parameter m has an impact on the acceleration performance. We noticed that a larger value of m contributes to using more information for estimating the inverse Jacobian, which decreases the required number of epochs for convergence. However, increasing m raises the computational cost significantly and leads to overfitting, which negatively affects the training process. Therefore, we experimentally choose $m = 8$ for our experiments.

3.2.2. Analysis of Anderson acceleration

This subsection summarizes the Anderson acceleration algorithm [46]. The following matrix of secants in x is built based on the outcomes of the previous m iterations recollected from memory in each epoch k .

$$\begin{aligned} \xi_i^x &= x_{i+1} - x_i \\ \Xi_k^x &= [\xi_{k-m_k}^x \cdots \xi_{k-1}^x] \\ \xi_i^F &= F(x_{i+1}) - F(x_i) \\ \Xi_k^F &= [\xi_{k-m_k}^F \cdots \xi_{k-1}^F] \end{aligned} \quad (10)$$

where m_k denotes the memory in iteration k . As it is stated in [45], the estimated Jacobian of F minimizing $\|B_k - I\|_F$ subject to the condition $B_k \Xi_k^x = \Xi_k^F$ can be written as

$$B_k = I + (\Xi_k^F - \Xi_k^x)(\Xi_k^{xT} \Xi_k^x)^{-1} \Xi_k^{xT} \quad (11)$$

$$B_k^{-1} = I + (\Xi_k^x - \Xi_k^F)(\Xi_k^{xT} \Xi_k^F)^{-1} \Xi_k^{xT} \quad (12)$$

Thus, the quasi-Newton iteration is defined as

$$x_{k+1} = x_k - B_k^{-1} F_k \quad (13)$$

However, if either Ξ_k^x or Ξ_k^F suffers from rank deficiency, the iteration may be unable to carry out. To deal with this issue, [47] fixed B_k using its limitation to span (Ξ_k^x) and its orthogonal complement. Thus, in Eq. (11) B_k can be built inductively beginning from $B_k^0 = I$ as

$$B_k^{i+1} = B_k^i + \frac{(\xi_{k-m_k+i}^F - B_k^i \xi_{k-m_k+i}^x) \hat{\xi}_{k-m_k+i}^{xT}}{\hat{\xi}_{k-m_k+i}^{xT} \xi_{k-m_k+i}^x} \quad (14)$$

where $i = 0, \dots, m_k - 1$ and $B_k = B_k^{m_k}$. In addition, $\{\hat{\xi}_i^x\}_{i=k-m_k}^{k-1}$ can be acquired from $\{\xi_i^x\}_{i=k-m_k}^{k-1}$ via applying Gram-Schmidt orthogonalization:

$$\hat{\xi}_i^x = \xi_i^x - \sum_{j=k-m_k}^{i-1} \frac{\hat{\xi}_j^{xT} \xi_i^x}{\hat{\xi}_j^{xT} \xi_j^x} \hat{\xi}_j^x, \quad i = k - m_k, \dots, k - 1. \quad (15)$$

Eq. (14) becomes singular when $\hat{\xi}_{k-m_k+i}^x = 0$ while $\xi_{k-m_k+i}^x \neq 0$. This issue can occur even if m_k is chosen as $\min\{m, k\}$ for some fixed $1 \leq m \leq \infty$, where m represents the full memory. In particular, when m exceeds the problem dimension n , $\hat{\xi}_k = 0$ for $k > n$ as a result of linear dependence. To address this problem, we implement a restart checking procedure that clears the memory just before the algorithm is close to stagnation. To be more distinct, m_k is increasing up to either $m_k = m + 1$ for some integer $1 \leq m \leq \infty$, or the Gram-Schmidt orthogonalization will be ill-conditioned $\|\hat{\xi}_{k-1}^x\|_2 < \tau \|\xi_{k-1}^x\|_2$ for $\tau \in (0, 1)$, in which circumstance m_k is reset to zero. Furthermore, to avoid the possible singularity of B_k , [47] defined $\bar{\theta} \in (0, 1)$ to replace $\xi_{k-m_k+i}^F$ in (14) with

$$\xi_{k-m_k+i}^F = \theta_k^i \xi_{k-m_k+i}^F + (1 - \theta_k^i) B_k^i \xi_{k-m_k+i}^x \quad (16)$$

$$\text{where } \theta_k^i = \psi_{\bar{\theta}} \left(\frac{\hat{\xi}_{k-m_k+i}^{xT} (B_k^{-1})^T \xi_{k-m_k+i}^F}{\|\hat{\xi}_{k-m_k+i}^x\|_2} \right), \text{ and } \psi_{\bar{\theta}}(\eta) \text{ is determined by}$$

$$\psi_{\bar{\theta}}(\eta) = \begin{cases} (1 - \text{sign}(\eta) \bar{\theta}) / (1 - \bar{\theta}) & \text{if } |\eta| < \bar{\theta} \\ 1 & \text{if } |\eta| \geq \bar{\theta} \end{cases} \quad (17)$$

Notice that we have $B_k^i = B_{k-m_k+i}$ as restart checking is employed to clear the memory and B_k is computed in (14). Furthermore, we can eliminate the requirement for keeping updates for B_k^i in the regularization by observing that $B_k^i \xi_{k-m_k+i}^x = B_{k-m_k+i} \xi_{k-m_k+i}^x = -B_{k-m_k+i} B_{k-m_k+i}^{-1} F_{k-m_k+i} = -F_{k-m_k+i}$.

Lemma 1 (Anderson Acceleration's Well-Conditioned [47, Lemma 3]). By choosing the m_k from the restart checking strategy and considering (16), we obtain $\|B_k\| \leq 3(1 + \bar{\theta} + \tau)^m / \tau^m - 2$ for all $k \geq 0$.

Therefore, the rank deficiency issue is thoroughly resolved. Additionally, the inverse of B_k remains bounded as we have

$$\|B_k^{-1}\|_2 \leq \frac{\left(3 \left(\frac{1+\bar{\theta}+\tau}{\tau}\right)^m - 2\right)^{n-1}}{\bar{\theta}^m}. \quad (18)$$

where n is the dimension of x .

Moreover, to guarantee the decline of the residual norm, F , we apply a safeguarding strategy developed by [47] to investigate that the residual norm is adequately small.

Finally, the Anderson acceleration approach for the proposed AA-mDLAM is summarized in Algorithm 1. Lines 4–7 implement the restart checking scheme, lines 8–11 execute the regularization, lines 12–13 perform the safeguarding method, and $r_{k+1} = z_{k+1} - W_{k+1} a_k - b_{k+1}$. Please notice that the Anderson acceleration recommends an update $\tilde{x}_k$ to ensure that $B_{k-1} \tilde{x}_{k-1} = -B_{k-1} B_{k-1}^{-1} F_{k-1} = -F_{k-1}$ is still retained, resulting in the maintenance of updating B_k being discarded. As a result, in line 4 of Algorithm 1, we applied $\tilde{x}_k$ instead of defining $\xi_{k-1}^x = x_k - x_{k-1}$ and $\xi_{k-1}^F = F(x_k) - F(x_{k-1})$.

Remark 1. To ensure a self-contained presentation and facilitate a fair and direct comparison with Refs. [11,15], the structure and problem formulation in the following section follows a similar approach. This alignment ensures consistency while allowing us to clearly distinguish the novel contributions and enhancements introduced by the proposed AA-mDLAM algorithm.

3.3. Algorithm updates

The proposed AA-mDLAM algorithm contains the following subproblems, which will be introduced in detail. The quadratic approximation and backtracking methods are employed to solve the subproblems to avoid matrix inversion.

1. Update W_l

The variables $W_l (l = 1, \dots, L)$ are updated as follows:

$$W_l^{k+1} \leftarrow \underset{W_l}{\operatorname{argmin}} \phi(a_{l-1}^{k+1}, W_l, z_l^k, b_l^k) + \Omega_l(W_l). \quad (19)$$

where $\phi(a_{l-1}^{k+1}, W_l, z_l^k, b_l^k) = \frac{\rho}{2} \|z_l - W_l a_{l-1} - b_l\|_2^2$. Due to the coupling of W_l and a_{l-1} in $\phi(\cdot)$, the inversion execution of a_{l-1}^{k+1} is required for solving W_l , which is computationally costly. To overcome this challenge, similarly to the dIADMM algorithm [11], we define $\mathcal{P}_l(W_l; \theta_l^{k+1})$ as a quadratic estimation of ϕ at W_l^k , which can be written as follows:

$$\begin{aligned} \mathcal{P}_l^{k+1}(W_l; \theta_l^{k+1}) &= \phi(a_{l-1}^k, \bar{W}_l^{k+1}, z_l^k, b_l^k) + (\nabla_{\bar{W}_l^{k+1}} \phi)^T \\ &\quad \left(W_l - \bar{W}_l^{k+1} \right) + \frac{\theta_l^{k+1}}{2} \|W_l - \bar{W}_l^{k+1}\|_2^2 \end{aligned} \quad (20)$$

Algorithm 1 Anderson Acceleration Algorithm

Require: Initialization: $x_{\text{default}} = x_0$, $r_{\text{prev}} = \infty$, $k = 0$, $\text{reset} = \text{TRUE}$, safeguarding constants $d, \varepsilon > 0$, $\alpha, \bar{\theta}, \tau \in (0, 1)$, $m_0 = n_{AA} = 0$, $\bar{U} = \|F(x_0)\|_2$, $x_1 = \bar{x}_1 = (1 - \alpha)x + \alpha G(x)$

1: **while** not converged **do**

2: **if** $\text{reset} = \text{TRUE}$ **OR** $r < r_{\text{prev}}$ **then**

 # Record the recent accepted iterate

3: $x_k = x_{\text{default}}$, $r_{\text{prev}} = r$, $\text{reset} = \text{FALSE}$

4: **end if**

 # Compute the accelerated iterate

5: $\xi_{k-1}^x = \bar{x}_k - x_{k-1}$, $\xi_{k-1}^F = F(\bar{x}_k) - F(x_{k-1})$

6: $\hat{\xi}_{k-1}^x = \xi_{k-1}^x - \sum_{j=k-m_k}^{k-2} \frac{\hat{\xi}_{j-1}^{xT} \xi_{k-1}^x}{\hat{\xi}_{j-1}^{xT} \xi_{j-1}^x} \xi_j^x$

7: **if** $m_k = m + 1$ **or** $\|\hat{\xi}_{k-1}^x\|_2 < \tau \|\xi_{k-1}^x\|_2$ **then**

8: Reset $m_k = 0$, $\hat{\xi}_{k-1}^x = \xi_{k-1}^x$, and $B_{k-1}^{-1} = I$

9: **end if**

10: Compute $\bar{\xi}_{k-1}^F = \theta_{k-1} \xi_{k-1}^F - (1 - \theta_{k-1}) F_{k-1}$

11: With $\theta_{k-1} = \psi_{\bar{\theta}}(\eta_{k-1})$, and

$\eta_{k-1} = \hat{\xi}_{k-1}^{xT} B_{k-1}^{-1} \xi_{k-1}^F / \|\hat{\xi}_{k-1}^x\|_2^2$

12: Update:

$B_k^{-1} = B_{k-1}^{-1} + \frac{(\xi_{k-1}^x - B_{k-1}^{-1} \bar{\xi}_{k-1}^F) \hat{\xi}_{k-1}^{xT} B_{k-1}^{-1}}{\hat{\xi}_{k-1}^{xT} B_{k-1}^{-1} \xi_{k-1}^F}$

13: Compute $\bar{x}_{k+1} = x_k - B_k^{-1} F(x_k)$

14: **if** $\|F(x_k)\| \leq d \bar{U} (n_{AA} + 1)^{-(1+\varepsilon)}$ **then**

15: $x_{k+1} = \bar{x}_{k+1}$, $n_{AA} = n_{AA} + 1$

16: **else**

 # Revert to the last accepted iterate

17: $\text{reset} = \text{TRUE}$, $x_k = x_{\text{default}}$

18: **end if**

19: Increment $k \leftarrow k + 1$

20: **end while**

21: **return** x_k

where $\bar{W}_l^{k+1} = W_l^k$ and $\theta_l^{k+1} > 0$ denotes a scalar parameter, which is adopted by the backtracking algorithm [11] to satisfy the condition below

$$P_l^{k+1}(W_l; \theta_l^{k+1}) \geq \phi(a_l^k, \bar{W}_l^{k+1}, z_l^k, b_l^k), \quad (21)$$

and $\nabla_{\bar{W}_l^{k+1}} \phi$ is the gradient of W_l at $\bar{W}_l^{k+1}$. Thus, we minimize the following rather than minimizing (19).

$$W_l^{k+1} \leftarrow \underset{W_l}{\operatorname{argmin}} P_l^{k+1}(W_l; \theta_l^{k+1}) + \Omega_l(W_l), \quad (22)$$

Eq. (22) holds a closed-form solution for assuming either l_1 or l_2 regularization.

2. Update a_l

The variables $a_l (l = 1, \dots, L - 1)$ are updated as follows:

$$a_l^{k+1} \leftarrow \underset{a_l}{\operatorname{argmin}} \phi(a_l, W_{l+1}^k, z_{l+1}^k, b_{l+1}^k) \quad (23)$$

s.t. $h_l(z_l^{k+1}) - \varepsilon \leq a_l \leq h_l(z_l^{k+1}) + \varepsilon$

Likewise, we introduce $Q_l^{k+1}(a_l; \tau_l^{k+1})$ as a quadratic approximation of ϕ at a_l^{k+1} . It can be written mathematically as follows

$$Q_l^{k+1}(a_l; \tau_l^{k+1}) = \phi(a_l^{k+1}, W_{l+1}^k, z_{l+1}^k, b_{l+1}^k) + (\nabla_{a_l^{k+1}} \phi)^T (a_l - a_l^{k+1}) + \frac{\tau_l^{k+1}}{2} \|a_l - a_l^{k+1}\|_2^2 \quad (24)$$

where τ_l^{k+1} is a scalar parameter selected by the backtracking strategy [11] to satisfy the following condition

$$Q_l^{k+1}(a_l; \tau_l^{k+1}) \geq \phi(a_l^{k+1}, W_{l+1}^k, z_{l+1}^k, b_{l+1}^k). \quad (25)$$

Therefore, we minimize the following problem instead of solving the minimization problem in (23)

$$a_l^{k+1} \leftarrow \underset{a_l}{\operatorname{argmin}} Q_l^{k+1}(a_l; \tau_l^{k+1}) \quad (26)$$

s.t. $h_l(z_l^{k+1}) - \varepsilon \leq a_l \leq h_l(z_l^{k+1}) + \varepsilon$

The solution to (26) is acquired as follows by assuming L^{k+1} and U_l^{k+1} to be the lower and upper bound of the constraint $\{h_l(z_l^{k+1}) - \varepsilon \leq a_l \leq h_l(z_l^{k+1}) + \varepsilon\}$, respectively.

$$a_l^{k+1} \leftarrow \min \left(\max(L^{k+1}, a_l^{k+1} - \nabla_{a_l^{k+1}} \phi / \tau_l^{k+1}), U_l^{k+1} \right). \quad (27)$$

3. Update z_l

The variables $z_l (l = 1, \dots, L - 1)$ are updated as follows:

$$z_l^{k+1} \leftarrow \underset{z_l}{\operatorname{argmin}} \phi(a_{l-1}^k, W_l^{k+1}, z_l, b_l^{k+1}) \quad (28)$$

s.t. $h_l(z_l^{k+1}) - \varepsilon \leq a_l \leq h_l(z_l^{k+1}) + \varepsilon$,

and for z_L^{k+1}

$$z_L^{k+1} \leftarrow \underset{z_L}{\operatorname{argmin}} \phi(a_{L-1}^{k+1}, W_L^{k+1}, z_L, b_L^{k+1}) + R(z_L; y). \quad (29)$$

Similarly, we define $\mathcal{V}_l^{k+1}(z_l)$ as follows:

$$\mathcal{V}_l^{k+1}(z_l) = \phi(a_{l-1}^{k+1}, W_l^{k+1}, z_l^{k+1}, b_l^{k+1}) + (\nabla_{z_l^{k+1}} \phi)^T (z_l - z_l^{k+1}) + \frac{\rho}{2} \|z_l - z_l^{k+1}\|_2^2 \quad (30)$$

where ρ is the penalty parameter. Thus, the following problems should be solved instead of (28) and (29).

$$z_l^{k+1} \leftarrow \underset{z_l}{\operatorname{argmin}} \mathcal{V}_l^{k+1}(z_l) \quad (31)$$

s.t. $h_l(z_l^{k+1}) - \varepsilon \leq a_l \leq h_l(z_l^{k+1}) + \varepsilon \quad (l < L),$

$$z_L^{k+1} \leftarrow \underset{z_L}{\operatorname{argmin}} \mathcal{V}_L^{k+1}(z_L) + R(z_L; y). \quad (32)$$

The solution for $z_l (l = 1, \dots, L - 1)$ is attained by

$$z_l^{k+1} \leftarrow \min \left(\max(L^{k+1}, z_l^{k+1} - \nabla_{z_l^{k+1}} \phi / \rho), U_l^{k+1} \right). \quad (33)$$

On account of the convexity of $\mathcal{V}(\bullet)$ and $R(\bullet)$ with respect to z_L , (32) is a convex problem. Accordingly, the Fast Iterative Soft-Thresholding Algorithm (FISTA) [48] can be used to solve (32).

4. Update b_l

The variables $b_l (l = 1, \dots, L)$ are updated as follows:

$$b_l^{k+1} \leftarrow \underset{b_l}{\operatorname{argmin}} \phi(a_{l-1}^{k+1}, W_l^{k+1}, z_l^{k+1}, b_l) \quad (34)$$

Similarly to update z_l^{k+1} , we define $\mathcal{U}_l^{k+1}(b_l; \rho)$ as a quadratic approximation of $\phi(\bullet)$ at b_l^k , which can be mathematically reformulated as follows:

$$\mathcal{U}_l^{k+1}(b_l; \rho) = \phi(a_{l-1}^{k+1}, W_l^{k+1}, z_l^{k+1}, b_l^{k+1}) + (\nabla_{b_l^{k+1}} \phi)^T (b_l - b_l^{k+1}) + \frac{\rho}{2} \|b_l - b_l^{k+1}\|_2^2. \quad (35)$$

Hence, the following problem is minimized instead of minimizing (34).

$$b_l^{k+1} \leftarrow \underset{b_l}{\operatorname{argmin}} \mathcal{U}_l^{k+1}(b_l; \rho). \quad (36)$$

The solution to (36) can be obtained as follows since it is convex and has a closed-form solution.

$$b_l^{k+1} \leftarrow b_l^{k+1} - \nabla_{b_l^{k+1}} \phi / \rho \quad (37)$$

The proposed AA-mDLAM approach is summarized in detail in Algorithm 2, which outlines the iterative update process for optimizing a_l , b_l , W_l , and z_l using Anderson acceleration. The efficiency and convergence properties of this algorithm will be further analyzed in the following section.

Algorithm 2 The proposed AA-mDLAM algorithm

Require: Input: $y, a_0 = x$. Output: a_l, W_l, z_l, b_l ($l = 1, \dots, L$). Initial parameters ($W_{default} = W_{(0)}, z_{default} = z_{(0)}, a_{default} = a_{(0)}, b_{default} = b_{(0)}$), Anderson memory m , maximum iterations K , and stopping threshold ϵ .

```

1: Initialize residual  $r_{prev} = +\infty$ , iteration counter  $k = 0$ , and Anderson
   acceleration variables  $j = 0$ ,  $reset = \text{TRUE}$ .
2: while  $a_l^{k+1}, W_l^{k+1}, z_l^{k+1}, b_l^{k+1}$  not converge do
3:   for  $l = 1$  to  $L$  do
     # Alternating Minimization Updates
4:     Update  $W_l^{k+1}$  in Eq. (22).
5:     Update  $a_l^{k+1}$  in Eq. (26).
6:     if  $l=L$  then
7:       Update  $z_l^{k+1}$  in Eq. (32).
8:     else
9:       Update  $z_l^{k+1}$  in Eq. (31).
10:    end if
11:    Update  $b_l^{k+1}$  in Eq. (36).
12:    Compute the residual:

$$r = \|F(W_{l-1}^{k+1}, z_{l-1}^{k+1}, a_{l-1}^{k+1}, b_{l-1}^{k+1}) - F(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1})\|$$

     # Check Anderson Acceleration Conditions
13:     if  $reset = \text{FALSE}$  and  $r \geq r_{prev}$  then
14:       Reset variables:
15:        $(W_{default}, z_{default}, a_{default}, b_{default}) =$ 
 $(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1})$ 
16:        $reset = \text{TRUE}$ 
17:     end if
     # Check Termination Criteria
18:     if  $k + 1 \geq K$  or  $r < \epsilon$  then
19:       return  $(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1})$ 
20:     end if
     # Apply Anderson extrapolation for  $(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1})$ 
21:     Compute Anderson-accelerated values:

$$(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1}) = AA$$

22:     Recover compatible  $W^{(k+1)}$  using updated variables:

$$W^{(k+1)} = \text{update rule for } W$$

23:     Update iteration counter:  $k = k + 1$ ,  $r_{prev} = r$ 
24:   end for
25:   Increment  $k \leftarrow k + 1$ .
26: end while
27: Output  $a, W_l, z_l, b_l$ .
```

4. Convergence analysis

In this section, we investigate the convergence of the proposed AA-mDLAM algorithm. The following assumptions are deemed for the convergence analysis of the AA-mDLAM algorithm.

Assumption 1. $F(W, z, a, b)$ is coercive over the domain $D = \{(W, z, a, b) | h_l(z_l) - \epsilon \leq a - l \leq h_l(z_l) + \epsilon, (l = 1, \dots, L - 1)\}$. To put it simply, if $(W, z, a, b) \in D$ and $\|(W, z, a, b)\| \rightarrow \infty$, then we have $F(W, z, a, b) \rightarrow \infty$. Furthermore, $R(z_l; y)$ is Lipschitz differentiable with Lipschitz constant $H \geq 0$.

Assumption 1 is a mild assumption as common loss functions, including the least square loss and the cross-entropy loss, satisfy this assumption.

Remark 2. The foundational elements of Lemmas 2–4, given below, follow the theoretical framework established in [15]. Specifically, the structure of these lemmas aligns with the convergence analysis presented in [15]. However, our approach extends this framework by incorporating Anderson acceleration, which refines the iterative updates and improves the convergence rate. This enhancement introduces a more efficient optimization process and accelerates convergence in deep neural network training.

Lemma 2. *There exist $\beta_l, \delta_l, \gamma_l, \zeta_l > 0$ for $k \in \mathbb{N}$ such that it holds*

$$F(W_{l-1}^{k+1}, z_{l-1}^{k+1}, a_{l-1}^{k+1}, b_{l-1}^{k+1}) - F(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1}) \geq \frac{\beta_l^{k+1}}{2} \|W_l^{k+1} - W_l^k\|_2^2. \quad (38)$$

$$F(W_l^{k+1}, z_{l-1}^{k+1}, a_{l-1}^{k+1}, b_{l-1}^{k+1}) - F(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1}) \geq \frac{\delta_l^{k+1}}{2} \|z_l^{k+1} - z_l^k\|_2^2. \quad (39)$$

$$F(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_{l-1}^{k+1}) - F(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1}) \geq \frac{\gamma_l^{k+1}}{2} \|a_l^{k+1} - a_l^k\|_2^2. \quad (40)$$

$$F(W_l^{k+1}, z_{l-1}^{k+1}, a_{l-1}^{k+1}, b_{l-1}^{k+1}) - F(W_l^{k+1}, z_l^{k+1}, a_l^{k+1}, b_l^{k+1}) \geq \frac{\zeta_l^{k+1}}{2} \|b_l^{k+1} - b_l^k\|_2^2. \quad (41)$$

Lemma 2 illustrates that the objective declines as all variables are updated. The following convergence properties hold concerning Assumption 1 and Lemma 2. A similar idea has been applied in [15] to prove this Lemma, and thus it is omitted.

Lemma 3 (Objective Decrease [49, Property 2]).

$F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1}) \leq F(W^k, z^k, a^k, b^k)$ for any $k \in \mathbb{N}$. Furthermore, $F(W^k, z^k, a^k, b^k) \rightarrow F^*$ as $k \rightarrow \infty$, (F^* is the convergent value of F).

Lemma 3 depicts the decrease in the objective value during the iterations.

Lemma 4. In AA-mDLAM algorithm, $F(W^k, z^k, a^k, b^k)$ is upper bounded for any $k \in \mathbb{N}$. Moreover, some scalars M_W, M_z, M_a , and M_b exist such that $\|W^k\| \leq M_W, \|z^k\| \leq M_z, \|a^k\| \leq M_a$, and $\|b^k\| \leq M_b$, which implies that (W^k, z^k, a^k, b^k) are bounded.

This lemma shows that all variables and the objective are bounded and $\lim_{k \rightarrow \infty} W^{k+1} - W^k = 0, \lim_{k \rightarrow \infty} z^{k+1} - z^k = 0, \lim_{k \rightarrow \infty} a^{k+1} - a^k = 0$, and $\lim_{k \rightarrow \infty} b^{k+1} - b^k = 0$. The reader can refer to [15] for the proof.

Lemma 5. For any $k \in \mathbb{N}$, there exist a constant $C > 0$, and $g_1^{k+1} \in \partial_{W^{k+1}} F((W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1}))$ such that

$$\|g_1^{k+1}\| \leq C \left(\|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right). \quad (42)$$

Lemma 5 indicates that the subgradient of the objective is bounded regarding its variables. On this account, the subgradient converges to 0; thereby, it converges to a stationary point [49]. The proof is provided in Appendix B.

Property 1 (Convergence of the Residual F). Using (18), we obtain for an arbitrary fixed point $y \in X$ of G that

$$\begin{aligned} \|x_{k+1} - y\|_2 &\leq \|x_k - y\|_2 + \|B_k^{-1} F_k\|_2 \\ &\leq \|x_k - y\|_2 + C_G \|F_k\| \\ &\leq \|x_k - y\|_2 + C_G d \bar{U} (n_{AA} + 1)^{-(1+\epsilon)}. \end{aligned} \quad (43)$$

Table 2
Statics of four benchmark datasets.

Dataset	Num node	Num edge	Num class	Num feature
Cora	2708	5429	7	1433
Pubmed	19717	44 338	3	500
Citeseer	3327	4732	6	3703
Coauthor_CS	18 333	81 894	15	6805

Then we obtain

$$\|x_k - y\|_2 \leq \|x_0 - y\|_2 + C_G d \bar{U} \sum_{n_{AA}=0}^{\infty} (n_{AA} + 1)^{-(1+\epsilon)} = E < \infty \quad (44)$$

Hence $\|x_k - y\|_2$ stays bounded for $k \geq 0$. By taking the square from inequality (43), we acquire

$$\|x_{k+1} - y\|_2^2 \leq \|x_k - y\|_2^2 + (C_G d \bar{U})^2 (n_{AA} + 1)^{-(2+2\epsilon)} + 2C_G d E \bar{U} (n_{AA} + 1)^{-(1+\epsilon)} \quad (45)$$

According to line 12 of Algorithm 1, we have $\lim_{k \rightarrow \infty} \|F_k\|_2 = 0$, and by defining $\epsilon_k = (C_G d \bar{U})^2 (n_{AA} + 1)^{-(2+2\epsilon)}$, we obtain

$$\|x_{k+1} - y\|_2^2 \leq \|x_k - y\|_2^2 + \epsilon_k \quad (46)$$

where $\epsilon_k \geq 0$ and $\sum_{k=0}^{\infty} \epsilon_k < \infty$.

We summarize the result of the aforementioned Lemmas and Property 1 in Theorem 1, which indicates the convergence of the proposed AA-mDLAM algorithm using Anderson acceleration.

Theorem 1 (Convergence Under Anderson Acceleration). Suppose that Assumption 1 holds. Under the safeguarding step of Anderson acceleration, for any $\rho > 0$ and $\epsilon > 0$, Algorithm 1 converges to a stationary point x^* . That is, $0 \in \partial F_{W^*}(W^*, z^*, a^*, b^*)$.

Remark 3. According to Theorem 1, the convergence of Algorithm 1 under Anderson acceleration to a stationary point is guaranteed regardless of the choice of ρ and ϵ . This represents an improvement over the dlADMM algorithm [14], which relies on setting the hyperparameter to a sufficiently large value for convergence.

Theorem 2 (*r*-linear Convergence Rate). In the AA-mDLAM algorithm, if F is a lower semicontinuous convex subanalytic function, then the convergence rate of (W^k, z^k, a^k, b^k) is *r*-linear.

The proof is given in Appendix C.

Remark 4. As stated in Theorem 2, F is assumed to be a lower semicontinuous convex subanalytic function, which is a less restrictive condition compared to the assumption in [15, Theorem 2], where F is required to be locally strongly convex. Thus, the condition in our work is more general and applicable to a wider range of real-world scenarios. Furthermore, according to Theorem 2, the convergence rate of our proposed algorithm is *r*-linear, representing a significant improvement over existing alternating minimization methods such as dlADMM [11], which exhibit a sublinear convergence rate of $O(1/k)$. While the mDLAM [15] algorithm improves upon dlADMM [11] by achieving a linear convergence rate through the incorporation of momentum, our approach further enhances the theoretical guarantees and convergence efficiency by leveraging the Anderson acceleration framework.

5. Experiments

In this section, we evaluate the proposed algorithm utilizing seven benchmark datasets. The proposed algorithm's efficiency, convergence, and performance are compared with state-of-the-art approaches. All experiments are executed on a 64-bit machine with Intel(R) Iris(R) Xeon processor and NVIDIA GeForce RTX 3060 Ti.

Table 3

Hyperparameter setting on four datasets: they were chosen based on training performance.

Method	Hyper_ parameters	Cora	Pubmed	Citeseer	Coauthor_CS
Proposed method	ρ	1×10^{-4}	0.045	1×10^{-3}	7×10^{-4}
mDLAM	ρ	1×10^{-3}	0.01	5×10^{-3}	1×10^{-4}
dlADMM	ρ	1×10^{-6}	1×10^{-6}	1×10^{-6}	1×10^{-6}
GD	α	0.01	0.01	0.01	0.01
Adadelata	α	0.01	0.1	0.01	0.05
Adagrad	α	5×10^{-3}	5×10^{-3}	0.01	5×10^{-3}
Adam	α	1×10^{-3}	5×10^{-4}	1×10^{-3}	1×10^{-3}

5.1. Experimental setup on graph-based datasets

In the following section, we explain the datasets and the structure of the networks utilized for our experiments.

5.1.1. Dataset

In this experiment, we first evaluate the performance of our model on four graph-based benchmark datasets. The datasets' statistics are shown in Table 2 and are summarized below:

Cora [50] This dataset comprises 2708 machine learning publications categorized into one of seven classes. The citation networks contain 5429 links and 1433 specific words in the dictionary.

Pubmed [50] The Pubmed dataset consists of 19717 scientific papers from PubMed databases concerning diabetes divided into one of three classes. The citation networks include 44 338 links and 500 distinct words in the dictionary.

Citeseer [50] The Citeseer dataset contains 3312 scientific publications gathered from the Tagged.com social network website. It is classified into six classes and consists of 4732 links and 3703 unique words.

Coauthor CS [51] The Coauthor CS dataset is derived from the Microsoft Academic Graph from the KDD Cup 2016 contest. The nodes represent the authors connected by an edge if they co-authored a publication. Here, the keywords of each author's papers are node features, and class labels denote each author's most active research areas.

5.1.2. Experimental settings

In the first experiment on the four benchmark datasets including Cora, Pubmed, Citeseer, and Coauthor_CS, we set up a fully connected neural network as the basic model, which consists of three layers with 100 hidden units each. We considered the Rectified Linear Unit (ReLU) as a nonlinear activation function for each neuron. The cross-entropy with softmax was chosen as the loss function. The number of the epoch was set to 200.

In this study, the GD and its variant and ADMM are the state-of-the-art optimizers and thus carry out as comparison approaches. The full batch dataset trained the models. All parameters were selected on account of the maximum training accuracy. The baselines are outlined as follows:

Gradient Descent (GD) [52]: The GD is the most recognized deep learning optimizer in which parameters are updated concerning their gradient. The convergence of this method has been substantially investigated in the literature [53].

Momentum [19]: Momentum is an extension of the gradient descent optimization algorithm that addresses local minima and the oscillation of noisy gradients by incorporating inertia in a search direction. It uses a fraction of the previous update in the current update, which helps to smooth out the optimization path and accelerate convergence.

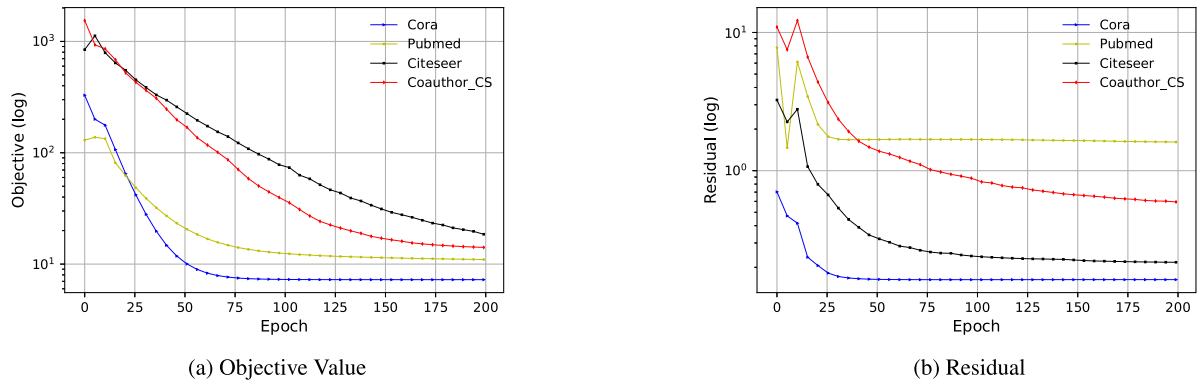


Fig. 1. Convergence curves of the proposed algorithm on four datasets: they all converge linearly when the number of iterations exceeds 100 roughly in (a) and 75 in (b).

Adaptive Gradient (Adagrad) [20]: It is a modified variant of the gradient descent optimization algorithm that enables the learning rate to be automatically adapted to the parameters.

Adaptive learning rate (Adadelata) [54]: It is an extension of the Adagrad to address the sensitivity to hyperparameter choice.

Adaptive momentum estimation (Adam) [21]: Adam is the most widespread deep learning optimization algorithm in generalizing proficiency. It approximates the first and second moments to improve the biased gradient, thereby accelerating the convergence rate.

Alternating Direction Method of Multipliers (ADMM) [8]: ADMM is an algorithm to solve convex optimization problems by breaking the objective function into smaller subproblems. Then, they cooperated in obtaining global solutions. This method can be adaptable to large-scale datasets and allows parallel computations.

The value of the hyperparameter used for all the comparison methods is depicted in Table 3. For the proposed algorithm and mDLAM method, the quadratic terms in Problem 3 are managed by ρ . For a fair comparison, we implemented mDLAM while incorporating the bias term b , as it is considered in its practical implementation, even though it is not explicitly mentioned in the theoretical formulation. In the dlADMM algorithm, ρ regulates a linear constraint and α is a learning rate regarding the other gradient-based methods. To relax the inequality constraints at the early phase in the proposed algorithm and mDLAM, ϵ is set adaptively to $\epsilon^{k+1} = \max(\frac{\epsilon^k}{2}, 0.001)$ with $\epsilon^0 = 100$.

5.1.3. Experimental results

In this section, we analyze the experimental outcomes of the proposed algorithm against the other state-of-the-art methods. The following criteria are investigated for comparison. (1) Convergence: the correlation between the number of iterations and objective value. (2) Performance: the test accuracy of all comparison methods on four benchmark datasets. (3) Running time: averaged one epoch running time respecting two parameters: the value of ρ and the number of hidden units. (4) Accuracy: the relationship between the maximum acquired test accuracy and the value of m and ρ . (5) Memory: memory usage of the proposed algorithm and mDLAM on all datasets.

5.1.3.1. Convergence. Fig. 1 depicts the convergence curves of the proposed algorithm on four datasets using the hyperparameters detailed in Table 3. In Fig. 1(a) and (b), the X -axis denotes the number of epochs and the Y -axis represents the logarithm of the objective value and residual, respectively. On all datasets, both the objective value and residual decline monotonously, which validates the convergence property of the proposed algorithm. Fig. 1(a) illustrates the trend differences in the objective curves. In the beginning, the objective

curves on the Cora and Pubmed datasets descend significantly and then attain the plateau after about 75 iterations, whilst the objective curves of the Coauthor CS reach the plateau when the iteration is roughly 150. The objective curve on the Citeseer maintains the downward trend over all iterations. Moreover, the residual and objective values on the Cora dataset are the lowest among other datasets at the end of the training. However, as shown in Fig. 1(b), the residual on the Pubmed dataset is around 1.6, which is five times more than other residuals of remaining datasets.

5.1.3.2. Performance. Fig. 2 shows the performance of the proposed method compared to the six state-of-the-art algorithms on four benchmark datasets. The X and Y axes denote the number of epochs and test accuracy, respectively. According to the experimental outcomes shown in Fig. 2, the proposed algorithm not only acquired the highest peak accuracy but also achieved convergence rapidly as against other comparison methods. As an illustration, the proposed algorithm attained 70% test accuracy in just 20 epochs on the Cora dataset, while mDLAM merely achieved 60%, GD gained 38% and Adadelata became flat around 43%. In addition, Fig. 2(a) depicts that the proposed algorithm greatly outperformed other approaches (approximately 80%). For the Citeseer dataset, the proposed algorithm is the superior optimizer, followed by the mDLAM algorithm, which is 2.3% lower in performance. In Fig. 2(d), the curves of the proposed algorithm notably surged to around 85% at the early stage and then increased steadily to 90% or more. However, other comparison methods, including GD and Adam, acquired half of their test accuracy at the 25th iteration (e.g., almost 40%). The worst algorithm is Adadelata, whose curves mount the slowest: its performance on the Cora, Pubmed, Citeseer, and Coauthor CS is 35%, 25%, 37%, and 53% inferior to the proposed algorithm, respectively. The mDLAM approach is on par with the proposed algorithm in some cases: the test accuracy of mDLAM substantially reaches that of the proposed algorithm on the Pubmed and Coauthor CS datasets. Due to the observation of the experimental results, the curves associated with the Adagrad algorithm on the Pubmed dataset and dlADMM on the Citeseer dataset drop at the end of epochs.

5.1.3.3. Running time. Next, we investigate the relationship between our proposed algorithm's running time per iteration and two factors: the number of hidden units and the number of ρ . The running time is averaged by 200 iterations.

Firstly, we analyze the effect of altering the number of hidden units on running time by increasing 100 neurons each time, ranging from 100 to 1000. Fig. 3 shows the computational results for all four datasets. Overall, the running time increased linearly along with the growth of the number of neurons. However, there are some cases: for instance, the running time per epoch dropped from 2.265 to 1.935 when the number of hidden units increased from 400 to 500 on the Citeseer dataset. The running times are under 1 s per epoch for all datasets except Coauthor CS, which is marginally more than 1 s (1.282 s) when the number of

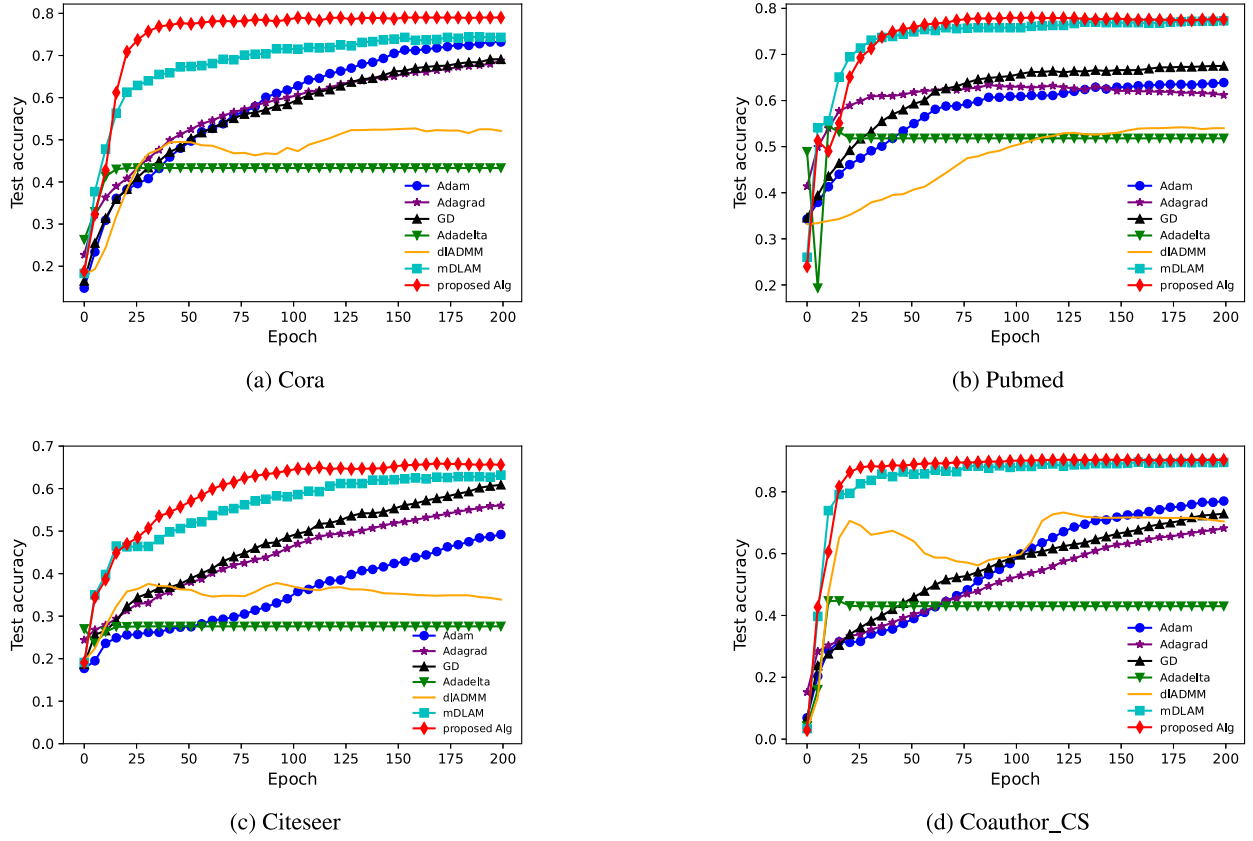


Fig. 2. Performance of all methods for four datasets: the proposed method outperforms all other algorithms.

neurons is 100. Nevertheless, the pace of increment differs on various datasets: the curve slope of the Coauthor CS soar sharply, which arrives at 12 s per epoch as the number of neurons is 1000, while the curves on the Pubmed and Cora rise subtly, which never exceed 1 and 2 s, respectively.

Secondly, we explore the running time per iteration by changing the value of ρ from 10^{-6} to 1 while the number of hidden units and m are fixed to 100 and 8 for each layer. Generally, the running time increase as the result of enlarging the value of ρ , as depicted in Fig. 4. The curve slope of Coauthor CS is superior, whereas the curve on the Pubmed, Cora, and Citeseer gradually climbs. Moreover, there exist some exceptions: the running time on Coauthor CS declines from 2.216 to 2.1164 s as the value of ρ enlarges from 10^{-2} to 10^{-1} , or the running time on the Pubmed drops from 0.215 to 0.092 s when ρ rises from 10^{-3} to 1.

5.1.3.4. Test accuracy. In this experiment, we evaluate the impact of hyperparameters, namely, the value of m and ρ on test accuracy. Table 4 describes the relation of test accuracy and the value of ρ , which alternated from 10^{-5} to 10^{-2} . In general, the proper choice of ρ plays a remarkable role in the test accuracy for all datasets. As an illustration, on the Pubmed dataset, when ρ enlarged from 10^{-3} to 10^{-2} , the performance enhanced to 0.776. However, the test accuracy on the Cora, Citeseer, and Coauthor CS datasets are comparably robust to the fluctuations of ρ from 10^{-5} to 10^{-3} . It should be mentioned that the performance of all datasets except Pubmed dropped dramatically when ρ was set to 10^{-2} , whereas the test accuracy of Pubmed improved the best to 0.776.

Table 5 demonstrates the correlation between test accuracy and the choice of m , which selects from 6, 8, 10, 12 on all datasets. Based on the obtained results, the performance is relatively resistant to the alteration of m . For instance, the test accuracy is near 0.90 regardless of the value of m on the Coauthor CS dataset. For the Pubmed dataset, the choice

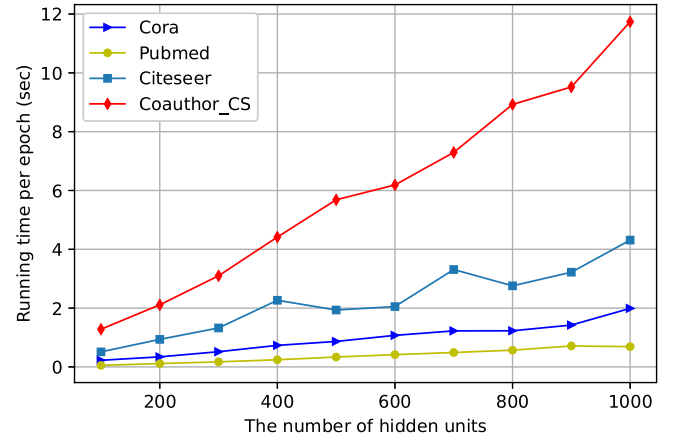


Fig. 3. The relationships between the running time per epoch (s) and the number of hidden units.

of m slightly affects test accuracy (around 0.12) when m changed from 12 to 8. In all experimental results, $m = 8$ achieved almost the best performance for all benchmark datasets. Comparing Tables 4 and 5, the value of ρ has a more significant impact on the test accuracy than that of m .

5.1.3.5. Memory analysis. Finally, we compare the memory usage of our proposed algorithm with the mDLAM method on all datasets as illustrated in Fig. 5. We can realize the following deduction due to the observation of memory consumption. First, the Coauthor CS utilizes the most memory, around 13 000 MB, while the Cora consumes

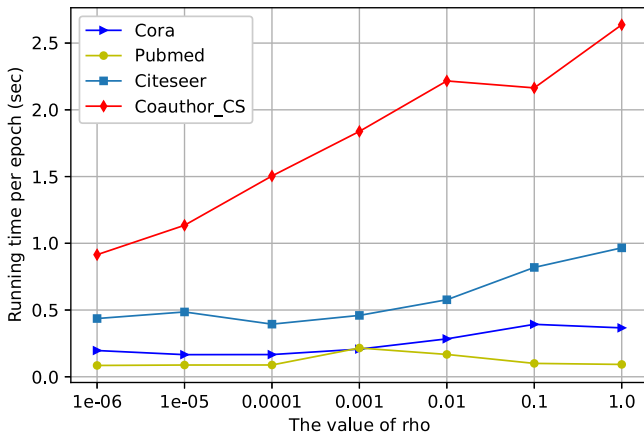


Fig. 4. The relationships between the running time per epoch (s) and the value of ρ .

Table 4

The relationship between test accuracy and the value of ρ .

(a) Cora					
ρ \ Epoch	40	80	120	160	200
10^{-5}	0.708	0.773	0.766	0.738	0.771
10^{-4}	0.772	0.78	0.783	0.783	0.783
10^{-3}	0.662	0.717	0.743	0.748	0.749
10^{-2}	0.298	0.271	0.27	0.263	0.258
(b) Pubmed					
ρ \ Epoch	40	80	120	160	200
10^{-5}	0.407	0.405	0.413	0.413	0.413
10^{-4}	0.4	0.399	0.4	0.401	0.402
10^{-3}	0.413	0.413	0.413	0.413	0.413
10^{-2}	0.752	0.782	0.779	0.776	0.776
(c) Citeseer					
ρ \ Epoch	40	80	120	160	200
10^{-5}	0.694	0.621	0.642	0.656	0.663
10^{-4}	0.637	0.656	0.66	0.657	0.658
10^{-3}	0.544	0.63	0.648	0.657	0.657
10^{-2}	0.262	0.237	0.233	0.233	0.233
(d) Coauthor_CS					
ρ \ Epoch	40	80	120	160	200
10^{-5}	0.885	0.896	0.894	0.894	0.894
10^{-4}	0.882	0.894	0.901	0.904	0.904
10^{-3}	0.826	0.865	0.866	0.874	0.876
10^{-2}	0.523	0.576	0.591	0.608	0.608

Table 5

The relationship between test accuracy and the choice of parameter m .

(a) Cora					
m \ Epoch	40	80	120	160	200
$m = 6$	0.67	0.72	0.742	0.749	0.752
$m = 8$	0.772	0.78	0.805	0.783	0.783
$m = 10$	0.772	0.785	0.79	0.79	0.79
$m = 12$	0.775	0.786	0.787	0.787	0.787
(b) Pubmed					
m \ Epoch	40	80	120	160	200
$m = 6$	0.753	0.775	0.778	0.779	0.778
$m = 8$	0.752	0.777	0.779	0.783	0.787
$m = 10$	0.754	0.779	0.775	0.776	0.778
$m = 12$	0.201	0.591	0.638	0.653	0.658
(c) Citeseer					
m \ Epoch	40	80	120	160	200
$m = 6$	0.539	0.634	0.653	0.655	0.656
$m = 8$	0.544	0.63	0.648	0.657	0.657
$m = 10$	0.55	0.627	0.645	0.652	0.655
$m = 12$	0.636	0.649	0.655	0.657	0.658
(d) Coauthor_CS					
m \ Epoch	40	80	120	160	200
$m = 6$	0.87	0.894	0.899	0.904	0.904
$m = 8$	0.882	0.894	0.901	0.904	0.904
$m = 10$	0.879	0.897	0.906	0.907	0.908
$m = 12$	0.879	0.901	0.907	0.908	0.907

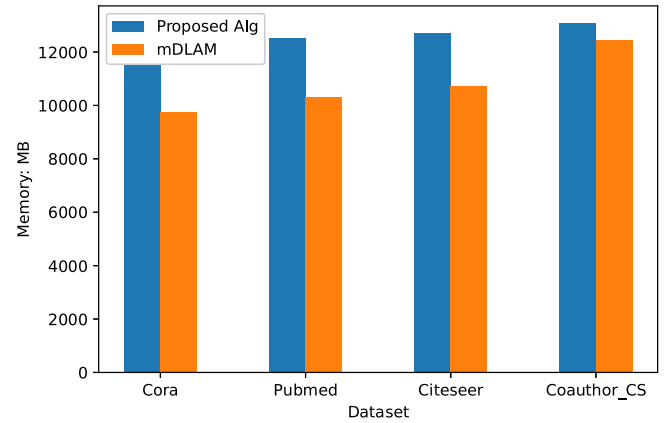


Fig. 5. Memory usage of the proposed Alg on all datasets versus mDLAM method.

approximately 12200 MB in the proposed algorithm. Second, comparing the four datasets, the average difference in memory usage of our proposed algorithm and mDLAM is around 1800 MB. Remarkably, our proposed algorithm occupies about 600 MB more than mDLAM on the Coauthor CS dataset, albeit exerting more iteration for an update. Overall, the computational outcomes demonstrate that the proposed algorithm occupies more memory while reaching better accuracy and converging rapidly rather than other comparison methods.

5.2. Experimental setup on image-based datasets

In the next section, we focus on image-based datasets and describe the datasets along with the network architectures used in our experiments.

5.2.1. Dataset

We evaluate our proposed algorithm on three well-known handwritten digits, fashion item datasets, and real-world image datasets. The datasets' statistics are shown in Table 6 and are summarized below:

MNIST [55]: The MNIST database is a large database of handwritten digits with 10 classes, which was created in 1994 as a combination of two of NIST's databases (<http://yann.lecun.com/exdb/mnist/>). It consists of 60,000 training and 10,000 testing 28×28 pixel grayscale images.

Fashion_MNIST [56]: The Fashion_MNIST dataset consists of 28×28 grayscale images of Zalando's article images, divided into 10 different clothing categories. It includes a training set of 60,000 images and a test set of 10,000 images.

CIFAR10 [57]: The CIFAR10 dataset consists of labeled subsets of 80 million tiny $32 \times 32 \times 3$ color images in 10 classes, which were collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. The dataset includes 50,000 training images and 10,000 test images.

5.2.2. Experimental settings

In this experiment, we designed a fully connected network for the MNIST and Fashion_MNIST datasets. This network began with an input

layer that flattened the 28×28 pixel grayscale images into a vector of 784 units. Following this, there were three fully connected hidden layers, each comprising 1000 neurons and using the ReLU activation function. The output layer consisted of 10 neurons, each corresponding to one of the 10 classes in the MNIST and Fashion_MNIST datasets, and employed a softmax activation function. The training was conducted over 200 epochs. The value of the hyperparameter used for all the comparison methods is depicted in Table 7.

Furthermore, we set up a network framework which comprised an input layer that processed the $32 \times 32 \times 3$ input images (flattened to 3072 units), followed by two fully connected hidden layers, each containing 1000 neurons, and an output layer with 10 neurons corresponding to the 10 classes in CIFAR10. Each hidden layer utilized the ReLU activation function. The training process used the entire dataset in each training step.

5.2.3. Experimental results

In this section, we investigate the experimental outcomes of the proposed algorithm against the other state-of-the-art methods. Fig. 6 depicts the test and training accuracy of our proposed AA-mDLAM algorithm alongside other comparison methods on the MNIST and Fashion_MNIST datasets. Overall, our results demonstrate that the AA-mDLAM algorithm achieves faster convergence and higher accuracy compared to the other optimization methods, particularly in the early stages of training as shown in Fig. 6(b), and Fig. 6(d). As an illustration, in Fig. 6(a), the AA-mDLAM algorithm attained about 88% test accuracy in just 20 epochs on the MNIST dataset, while SGD merely achieved 85%, Adam gained 60%, and Adadelata reached about 35%. Furthermore, in Fig. 6(c), the curves of the AA-mDLAM method climb to nearly 80% at the early stage, while other comparison algorithms, including Adadelata and Momentum gained approximately 52% and 68% at the 25th iteration, respectively.

The test accuracy results depicted in Fig. 7(a), show that the AA-mDLAM method significantly outperforms the other optimization methods, achieving higher accuracy (approximately 43%) more rapidly and maintaining superior performance across the epochs. The training accuracy outcomes shown in Fig. 7(b) similarly demonstrate that AA-mDLAM reaches a higher training accuracy, around 50%, much faster. In contrast, other comparison methods such as Adagrad, SGD, and Adam achieved only about 20% training accuracy at the 20th epoch. It should be noted that the training process utilizes the full batch of datasets.

As it can be observed in Fig. 7, the results on the full batch dataset for CIFAR10 were not satisfactory due to the low test and training accuracy and overall performance. Therefore, we employed our algorithm using minibatch size training and modified the network architecture to attain better results. The network structure for methods such as Stochastic Gradient Descent (SGD), Adadelata, Adagrad, Adam, and AA-mDLAM using TensorFlow and Keras consists of an input layer that flattens the $32 \times 32 \times 3$ images, followed by five fully connected hidden layers with 1024, 512, 256, 128, and 64 neurons, respectively. Each hidden layer utilizes the ReLU activation function and includes dropout regularization with a rate of 0.3 to mitigate overfitting. The output layer is a dense layer with 10 neurons, corresponding to the 10 classes in CIFAR10, using softmax activation. The training process spans 200 epochs, with a batch size of 64.

Fig. 8(a) illustrates the test accuracy of 200 epochs, where it is evident that all methods eventually achieve similar test accuracies, with AA-mDLAM, Adam, Adadelata, and Adagrad converging to around 55%. Fig. 8(b) presents the training accuracy over the same period, showing that AA-mDLAM not only converges significantly faster but also reaches higher training accuracy approximately 70% much earlier compared to the other methods and then continues to achieve around 80% and more. Adam also shows good convergence speed, followed by Adadelata and Adagrad, while SGD exhibits the slowest convergence rate.

Table 6

Statics of three image benchmark datasets.

Dataset	Num training	Num test	Num class	Num feature
MNIST	60,000	10,000	10	784
Fashion_MNIST	60,000	10,000	10	784
CIFAR10	50,000	10,000	10	3072

Table 7

Hyperparameter setting on MNIST, Fashion_MNIST, and CIFAR10 datasets: they were chosen based on training performance.

Method	Hyper_parameters	MNIST	Fashion_MNIST	CIFAR10
Proposed method	ρ	1×10^{-6}	1×10^{-6}	1×10^{-6}
SGD	ρ	1×10^{-4}	1×10^{-4}	1×10^{-5}
Adadelata	α	0.1	0.1	0.1
Adagrad	α	1×10^{-3}	1×10^{-3}	1×10^{-3}
Adam	α	1×10^{-4}	1×10^{-4}	1×10^{-3}
Momentum	α	1×10^{-6}	1×10^{-6}	–

6. Conclusion

The Alternating Direction Method of Multipliers (ADMM) is a superb alternative to Gradient Descent (GD) based methods for deep learning problems. In this study, we propose an efficient optimization algorithm named AA-mDLAM, which integrates Anderson acceleration into an Alternating Minimization framework inspired by ADMM. The Anderson acceleration approach is employed to elevate the convergence of the proposed AA-mDLAM method. Furthermore, the AA-mDLAM algorithm reaches a linear convergence, which is theoretically superior to other existing alternating optimizers. The experimental results on seven benchmark datasets indicate not only the proficiency and convergence of our proposed AA-mDLAM algorithm but also outperform the other state-of-the-art optimizers.

In future work, firstly we aim to extend our AA-mDLAM algorithm to convolutional neural networks (CNNs), particularly for datasets like CIFAR10, which typically achieve higher accuracy with CNN architectures. The CIFAR10 dataset contains $32 \times 32 \times 3$ color images, making it well-suited for convolutional operations that can capture spatial hierarchies and local patterns more effectively than fully connected layers.

Secondly, we can focus on developing adaptive methods for hyperparameter tuning. Such methods will aim to automatically adjust the hyperparameters ρ and ϵ during the training process, thereby enhancing the robustness and usability of the algorithm without extensive manual intervention.

CRedit authorship contribution statement

Zeinab Ebrahimi: Writing – original draft, Visualization, Software, Methodology, Investigation, Conceptualization. **Gustavo Batista:** Writing – review & editing, Validation, Supervision. **Mohammad Deghat:** Writing – review & editing, Validation, Supervision.

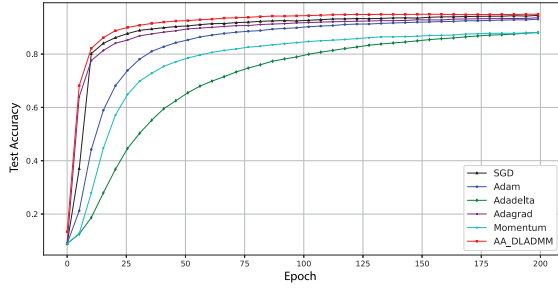
Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

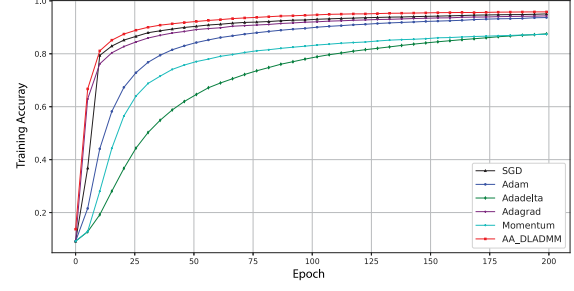
Appendix A. Proof of Lemma 3

By adding (38), (39), (40), and (41) for $l = 1, \dots, L$, we acquire

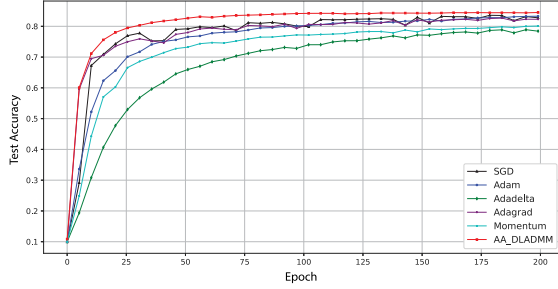
$$\begin{aligned} & \mathcal{F}(\mathbf{W}^k, \mathbf{z}^k, \mathbf{a}^k, \mathbf{b}^k) - \mathcal{F}(\mathbf{W}^{k+1}, \mathbf{z}^{k+1}, \mathbf{a}^{k+1}, \mathbf{b}^{k+1}) \geq \\ & \sum_{l=1}^L \left(\frac{\beta_l^{k+1}}{2} \|\mathbf{W}_l^{k+1} - \mathbf{W}_l^k\|_2^2 + \frac{\delta_l^{k+1}}{2} \|\mathbf{z}_l^{k+1} - \mathbf{z}_l^k\|_2^2 \right) \end{aligned}$$



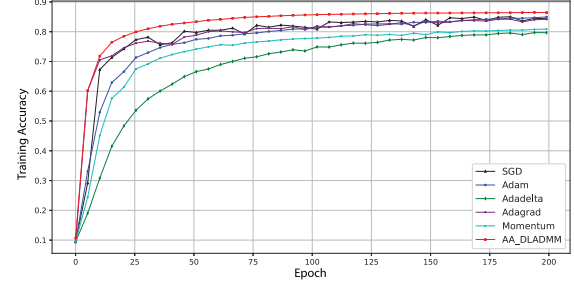
(a) Test Accuracy-MNIST



(b) Train Accuracy-MNIST

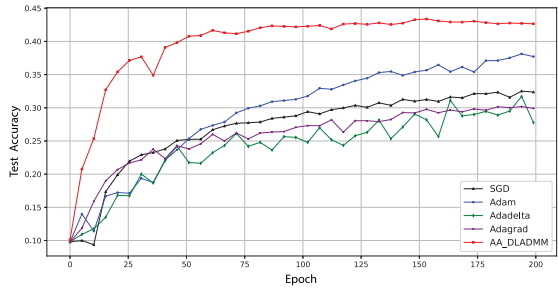


(c) Test Accuracy-Fashion_MNIST

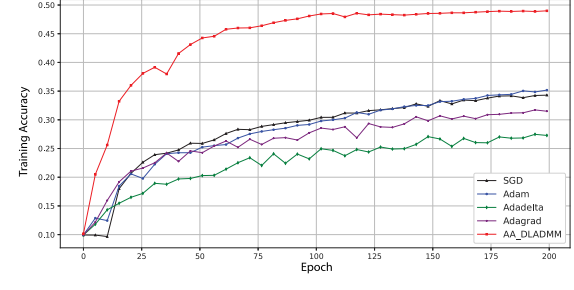


(d) Train Accuracy-Fashion_MNIST

Fig. 6. Performance of all methods on MNIST and Fashion_MNIST datasets: the proposed method (AA-mDLAM) outperforms all other algorithms.



(a) Test Accuracy-Cifar10



(b) Train Accuracy-Cifar10

Fig. 7. Performance of all methods on the CIFAR10 dataset.

$$+ \frac{\gamma_l^{k+1}}{2} \|b_l^{k+1} - b_l^k\|_2^2) + \sum_{l=1}^{L-1} \frac{\gamma_l^{k+1}}{2} \|a_l^{k+1} - a_l^k\|_2^2. \quad (47)$$

Let $Cs = \min(\frac{\beta_l^{k+1}}{2}, \frac{\delta_l^{k+1}}{2}, \frac{\zeta_l^{k+1}}{2}, \frac{\gamma_l^{k+1}}{2}) > 0$, hence

$$\begin{aligned} & \mathcal{F}(\mathbf{W}^k, \mathbf{z}^k, \mathbf{a}^k, \mathbf{b}^k) - \mathcal{F}(\mathbf{W}^{k+1}, \mathbf{z}^{k+1}, \mathbf{a}^{k+1}, \mathbf{b}^{k+1}) \\ & \geq Cs \left(\sum_{l=1}^L (\|W_l^{k+1} - W_l^k\|_2^2 + \|z_l^{k+1} - z_l^k\|_2^2 \right. \\ & \quad \left. + \|b_l^{k+1} - b_l^k\|_2^2) + \sum_{l=1}^{L-1} \|a_l^{k+1} - a_l^k\|_2^2 \right) \\ & = Cs (\|W^{k+1} - W^k\|_2^2 + \|z^{k+1} - z^k\|_2^2 \\ & \quad + \|b^{k+1} - b^k\|_2^2 + \|a^{k+1} - a^k\|_2^2) \geq 0. \end{aligned} \quad (48)$$

Owing to the boundedness of objectives and variables and the fact that the monotone sequence converges if it is bounded, then $\mathcal{F}(\mathbf{W}^k, \mathbf{z}^k, \mathbf{a}^k, \mathbf{b}^k)$ will converge.

Appendix B. Proof of Lemma 5

As we have

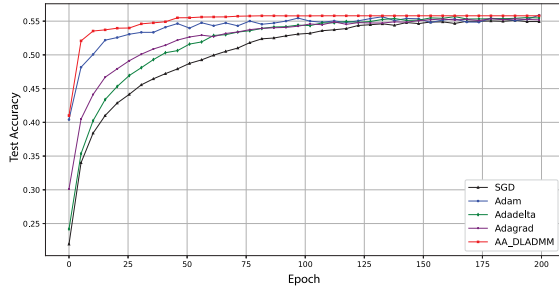
$$\mathcal{F}(\mathbf{W}_l^{k+1}, \mathbf{z}_{l-1}^{k+1}, \mathbf{a}_{l-1}^{k+1}, \mathbf{b}_{l-1}^{k+1}) < \mathcal{F}(\mathbf{W}^{k+1}, \mathbf{z}^{k+1}, \mathbf{a}^{k+1}, \mathbf{b}^{k+1}) \quad (49)$$

We require a linear combination of $\|\mathbf{W}^{k+1} - \mathbf{W}^k\|_2^2$,

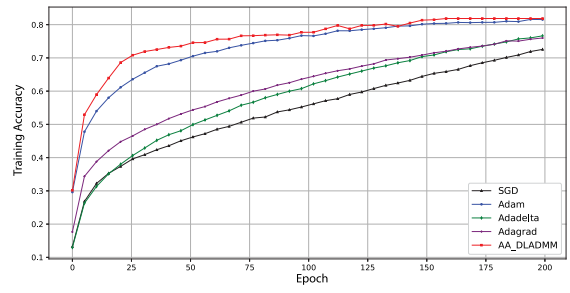
$\|\mathbf{z}^{k+1} - \mathbf{z}^k\|_2^2$, and $\|\mathbf{b}^{k+1} - \mathbf{b}^k\|_2^2$ as an upper bound to prove Lemma 5.

First for $\mathbf{W}^{k+1}$, according to the definition of $\mathcal{F}$ in Problem 3, we have

$$\begin{aligned} \partial_{W^{k+1}} \mathcal{F} &= \partial \Omega_l(W^{k+1}) + \nabla_{W^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) \\ &= \partial \Omega_l(W^{k+1}) + \nabla_{W_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) \\ &\quad + \nabla_{\bar{W}_l^{k+1}} \phi(\bar{W}_l^{k+1}, z_l^{k+1}, a_{l-1}^k, b_l^k) + \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) \\ &\quad - \nabla_{\bar{W}_l^{k+1}} \phi(\bar{W}_l^{k+1}, z_l^{k+1}, a_{l-1}^k, b_l^k) - \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) \\ &= \partial \Omega_l(W^{k+1}) + \nabla_{\bar{W}_l^{k+1}} \phi(\bar{W}_l^{k+1}, z_l^{k+1}, a_{l-1}^k, b_l^k) \\ &\quad + \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) + \rho (W_l^{k+1} - \bar{W}_l^{k+1}) a_{l-1}^{k+1} (a_{l-1}^{k+1})^\top \\ &\quad - \rho (z_l^{k+1} - z_l^k) (a_{l-1}^{k+1})^\top - \rho (b_l^{k+1} - b_l^k) (a_{l-1}^{k+1})^\top \\ &\quad - \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) \end{aligned} \quad (50)$$



(a) Test Accuracy-Cifar10



(b) Train Accuracy-Cifar10

Fig. 8. Performance of all methods on the CIFAR10 dataset with minibatch setting.

By applying triangle inequality, we can obtain

$$\begin{aligned} & \left\| \rho (W_l^{k+1} - \bar{W}_l^{k+1}) a_{l-1}^{k+1} (a_{l-1}^{k+1})^\top - \rho (z_l^{k+1} - z_l^k) (a_{l-1}^{k+1})^\top \right. \\ & \quad \left. - \rho (b_l^{k+1} - b_l^k) (a_{l-1}^{k+1})^\top - \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) \right\| \\ & \leq \rho \left\| (W_l^{k+1} - \bar{W}_l^{k+1}) a_{l-1}^{k+1} (a_{l-1}^{k+1})^\top \right\| + \rho \left\| (z_l^{k+1} - z_l^k) (a_{l-1}^{k+1})^\top \right\| \\ & \quad + \rho \left\| (b_l^{k+1} - b_l^k) (a_{l-1}^{k+1})^\top \right\| + \theta_l^{k+1} \left\| W_l^{k+1} - \bar{W}_l^{k+1} \right\| \end{aligned} \quad (51)$$

Now according to Cauchy-Schwartz inequality, we have

$$\begin{aligned} & \left\| \rho (W_l^{k+1} - \bar{W}_l^{k+1}) a_{l-1}^{k+1} (a_{l-1}^{k+1})^\top - \rho (z_l^{k+1} - z_l^k) (a_{l-1}^{k+1})^\top \right. \\ & \quad \left. - \rho (b_l^{k+1} - b_l^k) (a_{l-1}^{k+1})^\top - \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) \right\| \\ & \leq \rho \left\| W_l^{k+1} - \bar{W}_l^{k+1} \right\| \left\| a_{l-1}^{k+1} \right\| \left\| (a_{l-1}^{k+1})^\top \right\| + \rho \left\| z_l^{k+1} - z_l^k \right\| \left\| (a_{l-1}^{k+1})^\top \right\| \\ & \quad + \rho \left\| b_l^{k+1} - b_l^k \right\| \left\| (a_{l-1}^{k+1})^\top \right\| + \theta_l^{k+1} \left\| W_l^{k+1} - \bar{W}_l^{k+1} \right\| \end{aligned}$$

Based on Lemma 4

$$\begin{aligned} & \leq (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - \bar{W}_l^{k+1} \right\| + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| \\ & \quad + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \end{aligned}$$

By exerting Anderson Acceleration, we have

$$\begin{aligned} & \leq (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - (W_l^k - B_k^{-1} (W_l^k - W_l^{k+1})) \right\| \\ & \quad + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \end{aligned}$$

Now we apply triangle inequality to obtain

$$\begin{aligned} & \leq (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - W_l^k \right\| + (\rho M_a^2 + \theta_l^{k+1}) \left\| B_k^{-1} (W_l^{k+1} - W_l^k) \right\| + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| \\ & \quad + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \end{aligned} \quad (52)$$

Then due to (18) and Cauchy-Schwarz inequality, we acquire

$$\begin{aligned} & \leq (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - W_l^k \right\| + (\rho M_a^2 + \theta_l^{k+1}) \left\| \left(3 \left(\frac{1+\bar{\theta}+\tau}{\tau} \right)^m - 2 \right)^{n-1} / \bar{\theta}^m \right\| \left\| W_l^{k+1} - W_l^k \right\| \\ & \quad + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \end{aligned} \quad (53)$$

In addition, the optimality condition of (22) results in

$$\begin{aligned} 0 \in \partial \Omega_l (W_l^{k+1}) + \nabla_{\bar{W}_l^{k+1}} \phi (\bar{W}_l^{k+1}, z_l^{k+1}, a_{l-1}^k, b_l^k) \\ + \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}). \end{aligned} \quad (54)$$

Let $CB = \left(3 \left(\frac{1+\bar{\theta}+\tau}{\tau} \right)^m - 2 \right)^{n-1} / \bar{\theta}^m$. Thus, there exists $g_1^{k+1} \in \partial_{W_l^{k+1}} F$ such that

$$\begin{aligned} \left\| g_1^{k+1} \right\| & \leq (\rho M_a^2 + \theta_l^{k+1}) (1 + CB) \left\| W_l^{k+1} - W_l^k \right\| \\ & \quad + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| + \rho M_a \left\| b_l^{k+1} - b_l^k \right\|. \end{aligned} \quad (55)$$

Therefore, by considering $C = \max((\rho M_a^2 + \theta_l^{k+1})(1 + CB), \rho M_a)$, we conclude that

$$\begin{aligned} \left\| g_1^{k+1} \right\| & \leq C \left(\left\| W^{k+1} - W^k \right\| + \left\| z^{k+1} - z^k \right\| + \left\| b^{k+1} - b^k \right\| \right). \end{aligned} \quad (56)$$

On the other hand, when $(\bar{W}_l^{k+1} = W_l^k)$, we have

$$\begin{aligned} & \left\| \rho (W_l^{k+1} - \bar{W}_l^{k+1}) a_{l-1}^{k+1} (a_{l-1}^{k+1})^\top - \rho (z_l^{k+1} - z_l^k) (a_{l-1}^{k+1})^\top \right. \\ & \quad \left. - \rho (b_l^{k+1} - b_l^k) (a_{l-1}^{k+1})^\top - \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}) \right\| \\ & \leq (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - \bar{W}_l^{k+1} \right\| + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| \\ & \quad + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \\ & = (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - W_l^k \right\| + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| \\ & \quad + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \quad (\text{Since } \bar{W}_l^{k+1} = W_l^k). \end{aligned} \quad (57)$$

Similarly, the optimality condition of (22) yields

$$\begin{aligned} 0 \in \partial \Omega_l (W_l^{k+1}) + \nabla_{\bar{W}_l^{k+1}} \phi (\bar{W}_l^{k+1}, z_l^{k+1}, a_{l-1}^k, b_l^k) \\ + \theta_l^{k+1} (W_l^{k+1} - \bar{W}_l^{k+1}). \end{aligned} \quad (58)$$

Then there exists $g_1^{k+1} \in \partial_{W_l^{k+1}} F$ such that

$$\begin{aligned} \left\| g_1^{k+1} \right\| & \leq (\rho M_a^2 + \theta_l^{k+1}) \left\| W_l^{k+1} - W_l^k \right\| \\ & \quad + \rho M_a \left\| z_l^{k+1} - z_l^k \right\| + \rho M_a \left\| b_l^{k+1} - b_l^k \right\| \end{aligned} \quad (59)$$

Thus, by combining (56) and (59), we have

$$\begin{aligned} \left\| g_1^{k+1} \right\| & \leq C \left(\left\| W^{k+1} - W^k \right\| + \left\| z^{k+1} - z^k \right\| + \left\| b^{k+1} - b^k \right\| \right). \end{aligned} \quad (60)$$

Appendix C. Proof of Theorem 2

In this proof, we use the Kurdyka-Lojasiewics (KL) property [58]. We say f satisfies the KL property at $x^* \in \text{dom}(\partial f)$, if there exist $\eta \in (0, \infty)$, a neighborhood U of x^* , and a continuous concave function $\varphi : [0, \eta) \rightarrow \mathbb{R}_+$ such that (i) $\varphi(0) = 0$, $\varphi \in \mathbb{C}^1(0, \eta)$, $\varphi'(s) > 0, \forall s \in (0, \eta)$, and (iv) for all $x \in U_\eta := U \cap \{z \in \mathbb{R}^n, f(x^*) < f(z) < f(x^*) + \eta\}$, the KL inequality holds

$$\varphi'(f(x) - f(x^*)) \text{dist}(0, \partial f(x)) \geq 1. \quad (61)$$

According to [59, Definition F.1], φ has the KL property at x^* with exponent ϑ , if φ is chosen from Lojasiewicz functions and satisfies $\varphi(x) = qx^{1-\vartheta}$ for $q > 0$ and $\vartheta \in [0, 1)$.

Firstly, we know that

$$\partial F(\mathbf{W}^{k+1}, \mathbf{z}^{k+1}, \mathbf{a}^{k+1}, \mathbf{b}^{k+1}) \\ = (\partial_{\mathbf{W}^{k+1}} F, \partial_{\mathbf{z}^{k+1}} F, \partial_{\mathbf{a}^{k+1}} F, \partial_{\mathbf{b}^{k+1}} F)$$

where $\partial_{\mathbf{W}^{k+1}} F = \{\partial_{W_l^{k+1}} F\}_{l=1}^L$, $\partial_{\mathbf{z}^{k+1}} F = \{\partial_{z_l^{k+1}} F\}_{l=1}^L$, $\partial_{\mathbf{a}^{k+1}} F = \{\partial_{a_l^{k+1}} F\}_{l=1}^L$, $\partial_{\mathbf{b}^{k+1}} F = \{\partial_{b_l^{k+1}} F\}_{l=1}^L$. To prove Theorem 2, we require to provide an upper bound of $\partial_{\mathbf{W}^{k+1}} F$, $\partial_{\mathbf{z}^{k+1}} F$, $\partial_{\mathbf{a}^{k+1}} F$, and $\partial_{\mathbf{b}^{k+1}} F$ by a linear combination of $\|\mathbf{W}^{k+1} - \mathbf{W}^k\|$, $\|\mathbf{z}^{k+1} - \mathbf{z}^k\|$, $\|\mathbf{a}^{k+1} - \mathbf{a}^k\|$, and $\|\mathbf{b}^{k+1} - \mathbf{b}^k\|$.

$$\text{For } b_l^{k+1}, \\ \partial_{b_l^{k+1}} F = \nabla_{b_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) \\ = \nabla_{b_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) \\ + \nabla_{\bar{b}_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, \bar{b}_l^{k+1}) + \rho(b_l^{k+1} - \bar{b}_l^{k+1}) \\ - \rho(b_l^{k+1} - \bar{b}_l^{k+1}) - \nabla_{\bar{b}_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, \bar{b}_l^{k+1}) \\ = \rho(W_l^{k+1} a_{l-1}^{k+1} + b_l^{k+1} - z_l^{k+1}) + \rho(b_l^{k+1} - \bar{b}_l^{k+1}) \\ - \rho(b_l^{k+1} - \bar{b}_l^{k+1}) - \rho(W_l^{k+1} a_{l-1}^{k+1} + \bar{b}_l^{k+1} - z_l^{k+1}) \\ + \nabla_{\bar{b}_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, \bar{b}_l^{k+1}) \\ = \nabla_{\bar{b}_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, \bar{b}_l^{k+1}) + \rho(b_l^{k+1} - \bar{b}_l^{k+1})$$

The optimality condition of (36) yields

$$0 \in \nabla_{\bar{b}_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, \bar{b}_l^{k+1}) + \rho(b_l^{k+1} - \bar{b}_l^{k+1}),$$

Therefore, there exists $g_3^{k+1} = \partial_{b_l^{k+1}} F$ such that

$$\|g_3^{k+1}\| = 0. \quad (62)$$

For z_l^{k+1} ($l < L$), since the solution to (31) can be simplified as follows:

$$z_l^{k+1} \leftarrow z_l^{k+1} - \nabla_{z_l^{k+1}} \phi / \rho. \quad (63)$$

Therefore, we have

$$\partial_{z_l^{k+1}} F = \nabla_{z_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) \\ = \nabla_{z_l^{k+1}} \phi(W_l^{k+1}, z_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) \\ - \nabla_{\bar{z}_l^{k+1}} \phi(W_l^{k+1}, \bar{z}_l^{k+1}, a_{l-1}^{k+1}, b_l^{k+1}) - \rho(z_l^{k+1} - \bar{z}_l^{k+1}) \\ = \rho(z_l^{k+1} - W_l^{k+1} a_{l-1}^{k+1} - b_l^{k+1}) - \rho(\bar{z}_l^{k+1} - W_l^{k+1} a_{l-1}^{k+1} - b_l^{k+1}) - \rho(z_l^{k+1} - \bar{z}_l^{k+1}) = 0.$$

For z_L^{k+1} ,

$$\partial_{z_L^{k+1}} F = \nabla_{z_L^{k+1}} \phi(W_L^{k+1}, z_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1}) + \partial R(z_L^{k+1}; y) \\ = \nabla_{z_L^{k+1}} \phi(W_L^{k+1}, z_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1}) + \partial R(z_L^{k+1}; y) \\ + \nabla_{\bar{z}_L^{k+1}} \phi(W_L^{k+1}, \bar{z}_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1}) + \rho(z_L^{k+1} - \bar{z}_L^{k+1}) \\ - \rho(z_L^{k+1} - \bar{z}_L^{k+1}) - \nabla_{\bar{z}_L^{k+1}} \phi(W_L^{k+1}, \bar{z}_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1})$$

By the optimality condition of (32)

$$0 \in \partial R(z_L^{k+1}; y) + \nabla_{\bar{z}_L^{k+1}} \phi(W_L^{k+1}, \bar{z}_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1}) \\ + \rho(z_L^{k+1} - \bar{z}_L^{k+1}).$$

Then we obtain

$$\partial_{z_L^{k+1}} F = \nabla_{z_L^{k+1}} \phi(W_L^{k+1}, z_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1}) \\ - \nabla_{\bar{z}_L^{k+1}} \phi(W_L^{k+1}, \bar{z}_L^{k+1}, a_{L-1}^{k+1}, b_L^{k+1}) - \rho(z_L^{k+1} - \bar{z}_L^{k+1}) \\ \rho(z_L^{k+1} - W_L^{k+1} a_{L-1}^{k+1} - b_L^{k+1}) - \rho(\bar{z}_L^{k+1} - W_L^{k+1} a_{L-1}^{k+1} - b_L^{k+1}) - \rho(z_L^{k+1} - \bar{z}_L^{k+1}) = 0.$$

Thus, there exists $g_4^{k+1} = \partial_{z_L^{k+1}} F$ such that

$$\|g_4^{k+1}\| = 0. \quad (64)$$

For a_l^{k+1}

$$\partial_{a_l^{k+1}} F = \nabla_{a_l^{k+1}} \phi(W_{l+1}^{k+1}, z_{l+1}^{k+1}, a_l^{k+1}, b_{l+1}^{k+1})$$

As the solution to (26) can be expressed as follows:

$$a_l^{k+1} \leftarrow \bar{a}_l^{k+1} - \nabla_{\bar{a}_l^{k+1}} \phi / \tau_l^{k+1}, \quad (65)$$

Hence we have

$$\partial_{a_l^{k+1}} F = \nabla_{a_l^{k+1}} \phi(W_{l+1}^{k+1}, z_{l+1}^{k+1}, a_l^{k+1}, b_{l+1}^{k+1}) \\ - \nabla_{\bar{a}_l^{k+1}} \phi(W_{l+1}^k, z_{l+1}^k, \bar{a}_l^{k+1}, b_{l+1}^k) - \tau_l^{k+1} (a_l^{k+1} - \bar{a}_l^{k+1}) \\ = \rho(W_{l+1}^{k+1})^\top (W_{l+1}^{k+1} a_l^{k+1} + b_{l+1}^{k+1} - z_{l+1}^{k+1}) - \rho(W_{l+1}^k)^\top \\ (W_{l+1}^k \bar{a}_l^{k+1} + b_{l+1}^k - z_{l+1}^k) - \tau_l^{k+1} (a_l^{k+1} - \bar{a}_l^{k+1}) \\ = \rho(W_{l+1}^{k+1})^\top W_{l+1}^{k+1} (a_l^{k+1} - \bar{a}_l^{k+1}) + \rho(W_{l+1}^{k+1})^\top \\ (W_{l+1}^{k+1} - W_{l+1}^k) \bar{a}_l^{k+1} + \rho(W_{l+1}^{k+1} - W_{l+1}^k)^\top W_{l+1}^k \bar{a}_l^{k+1} \\ - \rho(W_{l+1}^{k+1})^\top (z_{l+1}^{k+1} - z_{l+1}^k) - \rho(W_{l+1}^{k+1} - W_{l+1}^k)^\top z_{l+1}^k \\ + \rho(W_{l+1}^{k+1})^\top (b_{l+1}^{k+1} - b_{l+1}^k) + \rho(W_{l+1}^{k+1} - W_{l+1}^k)^\top b_{l+1}^k \\ - \tau_l^{k+1} (a_l^{k+1} - \bar{a}_l^{k+1})$$

By applying Triangle inequality and Cauchy-Schwarz inequality and using Lemma 4, we acquire

$$\|\partial_{a_l^{k+1}} F\| \leq \rho \|W_{l+1}^{k+1}\| \|W_{l+1}^{k+1}\| \|a_l^{k+1} - \bar{a}_l^{k+1}\| \\ + \rho \|W_{l+1}^{k+1}\| \|W_{l+1}^{k+1} - W_{l+1}^k\| \|\bar{a}_l^{k+1}\| \\ + \rho \|W_{l+1}^{k+1} - W_{l+1}^k\| \|W_{l+1}^k\| \|\bar{a}_l^{k+1}\| \\ + \rho \|W_{l+1}^{k+1}\| \|z_{l+1}^{k+1} - z_{l+1}^k\| \\ + \rho \|W_{l+1}^{k+1} - W_{l+1}^k\| \|z_{l+1}^k\| \\ + \rho \|W_{l+1}^{k+1}\| \|b_{l+1}^{k+1} - b_{l+1}^k\| \\ + \rho \|W_{l+1}^{k+1} - W_{l+1}^k\| \|b_{l+1}^k\| \\ + \tau_l^{k+1} \|a_l^{k+1} - \bar{a}_l^{k+1}\| \\ \|\partial_{a_l^{k+1}} F\| \leq \rho M_W^2 \|a_l^{k+1} - \bar{a}_l^{k+1}\| + \rho M_W \|W_{l+1}^{k+1} - W_{l+1}^k\| \\ M_a + \rho \|W_{l+1}^{k+1} - W_{l+1}^k\| M_W M_a + \rho M_W \\ \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho \|W_{l+1}^{k+1} - W_{l+1}^k\| M_z \\ + \rho M_W \|b_{l+1}^{k+1} - b_{l+1}^k\| + \rho \|W_{l+1}^{k+1} - W_{l+1}^k\| M_b \\ + \tau_l^{k+1} \|a_l^{k+1} - \bar{a}_l^{k+1}\| \\ \leq (\rho M_W^2 + \tau_l^{k+1}) \|a_l^{k+1} - \bar{a}_l^{k+1}\| + (2\rho M_W M_a \\ + \rho M_z + \rho M_b) \|W_{l+1}^{k+1} - W_{l+1}^k\| + \rho M_W \\ \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho M_W \|b_{l+1}^{k+1} - b_{l+1}^k\|$$

According to the Anderson acceleration, we obtain

$$\|\partial_{a_l^{k+1}} F\| \leq (\rho M_W^2 + \tau_l^{k+1}) \|a_l^{k+1} - (a_l^k - B_k^{-1} (a_l^k - a_{l+1}^{k+1}))\| \\ + (2\rho M_W M_a + \rho M_z + \rho M_b) \|W_{l+1}^{k+1} - W_{l+1}^k\| \\ + \rho M_W \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho M_W \|b_{l+1}^{k+1} - b_{l+1}^k\| \\ \leq (\rho M_W^2 + \tau_l^{k+1}) \|a_l^{k+1} - a_l^k\| + (\rho M_W^2 + \tau_l^{k+1}) \\ \|B_k^{-1} (a_l^{k+1} - a_l^k)\| + (2\rho M_W M_a + \rho M_z + \rho M_b)$$

$$\begin{aligned}
& \|W_{l+1}^{k+1} - W_{l+1}^k\| + \rho M_W \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho M_W \\
& \|b_{l+1}^{k+1} - b_{l+1}^k\| \text{ (Triangle inequality)} \\
& \leq (\rho M_W^2 + \tau_l^{k+1}) \|a_l^{k+1} - a_l^k\| + (\rho M_W^2 + \tau_l^{k+1}) \\
& CB \|a_l^{k+1} - a_l^k\| + (2\rho M_W M_a + \rho M_z + \rho M_b) \\
& \|W_{l+1}^{k+1} - W_{l+1}^k\| + \rho M_W \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho M_W \\
& \|b_{l+1}^{k+1} - b_{l+1}^k\| \text{ ((18) and Cauchy-Schwarz inequality)}
\end{aligned}$$

Thus, there exists $g_5^{k+1} \in \partial_{a_l^{k+1}} F$ such that

$$\begin{aligned}
\|g_5^{k+1}\| & \leq (\rho M_W^2 + \tau_l^{k+1}) (1 + CB) \|a_l^{k+1} - a_l^k\| \\
& + (2\rho M_W M_a + \rho M_z + \rho M_b) \|W_{l+1}^{k+1} - W_{l+1}^k\| \\
& + \rho M_W \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho M_W \|b_{l+1}^{k+1} - b_{l+1}^k\|.
\end{aligned} \quad (66)$$

Otherwise if $(\bar{a}_l^{k+1} = a_l^k)$, then we have

$$\begin{aligned}
\|g_5^{k+1}\| & \leq (\rho M_W^2 + \tau_l^{k+1}) \|a_l^{k+1} - a_l^k\| \\
& + (2\rho M_W M_a + \rho M_z + \rho M_b) \|W_{l+1}^{k+1} - W_{l+1}^k\| \\
& + \rho M_W \|z_{l+1}^{k+1} - z_{l+1}^k\| + \rho M_W \|b_{l+1}^{k+1} - b_{l+1}^k\|.
\end{aligned} \quad (67)$$

Therefore, by defining $C_1 = \max((\rho M_W^2 + \tau_l^{k+1})(1 + CB), (2\rho M_W M_a + \rho M_z + \rho M_b), \rho M_W)$ and combining (66) and (67), we infer that

$$\begin{aligned}
\|g_5^{k+1}\| & \leq C_1 \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| \right. \\
& \left. + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right).
\end{aligned} \quad (68)$$

According to Lemma 5, (62), (64), and (68), we prove that there exists $g^{k+1} \in \partial F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1}) = (\partial_{W^{k+1}} F, \partial_{z^{k+1}} F, \partial_{a^{k+1}} F, \partial_{b^{k+1}} F)$ and $C_2 = \max(C, C_1, \rho)$ such that

$$\begin{aligned}
\|g^{k+1}\| & \leq C_2 \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| \right. \\
& \left. + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right).
\end{aligned} \quad (69)$$

Finally, we prove the convergence rate of the proposed algorithm by KL property. Let κ and η be the constants used in the KL property. By Lemma 3, $F(W^k, z^k, a^k, b^k) \rightarrow F^*$, then for any $\eta > 0$ and $k \geq k'$, we have $F^* < F(W^k, z^k, a^k, b^k) < F^* + \eta$. Define $\kappa_k = \varphi(F(W^k, z^k, a^k, b^k) - F^*)$. We employed the concavity property of κ for $k \geq k'$ as follows

$$\begin{aligned}
\kappa_k - \kappa_{k+1} & \geq \varphi'(\kappa_k) (F(W^k, z^k, a^k, b^k) - F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1})) \\
& \geq \frac{F(W^k, z^k, a^k, b^k) - F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1})}{\text{dist}(0, \partial F(W^k, z^k, a^k, b^k))}
\end{aligned} \quad (70)$$

Then, we require to obtain $\text{dist}(0, \partial F(W^k, z^k, a^k, b^k))$. Due to (69), we have

$$\begin{aligned}
\text{dist}(0, \partial F(W^k, z^k, a^k, b^k)) & = \|g^{k+1}\| \leq C_2 \left(\|a^{k+1} - a^k\| \right. \\
& \left. + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right).
\end{aligned} \quad (71)$$

Now according to Lemma 3 and the obtained result in (48), we have

$$\begin{aligned}
F(W^k, z^k, a^k, b^k) - F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1}) & \geq \\
C_S \left(\|W^{k+1} - W^k\|^2 + \|z^{k+1} - z^k\|^2 + \|b^{k+1} - b^k\|^2 \right. \\
& \left. + \|a^{k+1} - a^k\|^2 \right).
\end{aligned} \quad (72)$$

Then by (71), (72) and mean inequality, we have

$$\begin{aligned}
\kappa_k - \kappa_{k+1} & \geq \frac{C_S}{C_2} \\
& \left(\frac{\|W^{k+1} - W^k\|^2 + \|z^{k+1} - z^k\|^2 + \|b^{k+1} - b^k\|^2 + \|a^{k+1} - a^k\|^2}{\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\|} \right) \\
& \frac{C_S \left(\|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| + \|a^{k+1} - a^k\| \right)^2}{4C_2 \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right)}
\end{aligned} \quad (73)$$

Let $\bar{C} = \frac{C_S}{4C_2}$, then we have

$$\begin{aligned}
\kappa_k - \kappa_{k+1} & \geq \bar{C} \left(\|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| \right. \\
& \left. + \|b^{k+1} - b^k\| + \|a^{k+1} - a^k\| \right)
\end{aligned} \quad (74)$$

Now by summing (74) from k' to ∞

$$\begin{aligned}
\kappa_{k'} & \geq \bar{C} \sum_{k=k'}^{\infty} \left(\|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| \right. \\
& \left. + \|b^{k+1} - b^k\| + \|a^{k+1} - a^k\| \right).
\end{aligned} \quad (75)$$

It should be mentioned that κ_k is positive and monotone decreasing. Derived from Lemma 4, every term in this infinite series converges to zero since the total of this infinite series is finite. This indicates that $\lim_{k \rightarrow \infty} W^{k+1} - W^k = 0$, $\lim_{k \rightarrow \infty} z^{k+1} - z^k = 0$, $\lim_{k \rightarrow \infty} a^{k+1} - a^k = 0$, and $\lim_{k \rightarrow \infty} b^{k+1} - b^k = 0$. Now, we require to satisfy that $F(W^k, z^k, a^k, b^k)$ has the KL property with the exponent $\vartheta \in (0, \frac{1}{2}]$. By assuming $\varphi(x) = qx^{1-\vartheta}$ and the KL property's definition (61) and (71)

$$\begin{aligned}
C_2 (q(1-\vartheta)) \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| \right. \\
& \left. + \|b^{k+1} - b^k\| \right) \geq (q(1-\vartheta)) \text{dist}(0, \partial F(W^k, z^k, a^k, b^k)) \\
& \geq (F(W^k, z^k, a^k, b^k) - F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1}))^{\vartheta}.
\end{aligned} \quad (76)$$

Thus we obtain

$$\begin{aligned}
\kappa_k & = q(F(W^k, z^k, a^k, b^k) - F(W^{k+1}, z^{k+1}, a^{k+1}, b^{k+1}))^{1-\vartheta} \\
& \leq q \left(C_2 (q(1-\vartheta)) \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| \right. \right. \\
& \left. \left. + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right) \right)^{\frac{1-\vartheta}{\vartheta}}
\end{aligned} \quad (77)$$

It is clear that $\left(C_2 (q(1-\vartheta)) (\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\|) \right)^{\frac{1-\vartheta}{\vartheta}}$ is monotone increasing when $\vartheta \in (0, \frac{1}{2}]$ if $\left(C_2 q (\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\|) \right) < 1$. Therefore, for any $k \geq k'$ we obtain

$$\begin{aligned}
\kappa_k & \leq C_3 \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| \right. \\
& \left. + \|b^{k+1} - b^k\| \right)
\end{aligned} \quad (78)$$

where C_3 is constant. Then, we can acquire the following outcome similar to the previous proof.

$$\begin{aligned}
\kappa_k - \kappa_{k'} & \geq C_4 \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| \right. \\
& \left. + \|b^{k+1} - b^k\| \right)
\end{aligned} \quad (79)$$

Then by summing (79) from k' to ∞ , we have

$$\kappa_{k'} \geq C_4 \sum_{k=k'}^{\infty} \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right) \quad (80)$$

Thus

$$C_3 \left(\|a^{k'+1} - a^{k'}\| + \|W^{k'+1} - W^{k'}\| + \|z^{k'+1} - z^{k'}\| + \|b^{k'+1} - b^{k'}\| \right) \geq C_4 \sum_{k=k'}^{\infty} \left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right) \quad (81)$$

Let $\mathcal{M}_k = \sum_{i=k}^{\infty} \left(\|a^{i+1} - a^i\| + \|W^{i+1} - W^i\| + \|z^{i+1} - z^i\| + \|b^{i+1} - b^i\| \right)$, then we have

$$C_3(\mathcal{M}_{k'} - \mathcal{M}_{k'+1}) \geq C_4 \mathcal{M}_{k'} \Rightarrow \mathcal{M}_{k'+1} \leq \frac{C_3 - C_4}{C_3} \mathcal{M}_{k'} \quad (82)$$

Therefore, it is deduced that $\mathcal{M}_k$ converge q-linearly and attain the r-linear convergence of $\left(\|a^{k+1} - a^k\| + \|W^{k+1} - W^k\| + \|z^{k+1} - z^k\| + \|b^{k+1} - b^k\| \right)$.

Data availability

Data will be made available on request.

References

- [1] B. Hanin, Which neural net architectures give rise to exploding and vanishing gradients? *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [2] R. Novak, Y. Bahri, D.A. Abolafia, J. Pennington, J. Sohl-Dickstein, Sensitivity and generalization in neural networks: an empirical study, 2018, arXiv preprint arXiv:1802.08760.
- [3] Q. Tran-Dinh, D. Liu, A new randomized primal-dual algorithm for convex optimization with fast last iterate convergence rates, *Optim. Methods Softw.* 38 (1) (2023) 184–217.
- [4] J. Bai, L. Jia, Z. Peng, A new insight on augmented Lagrangian method with applications in machine learning, *J. Sci. Comput.* 99 (2) (2024) 53.
- [5] J.-C. Bai, F.-M. Bian, X.-K. Chang, L. Du, Accelerated stochastic peaceman-rachford method for empirical risk minimization, *J. Oper. Res. Soc. China* 11 (4) (2023) 783–807.
- [6] S. Na, M. Ma, S. Ma, G. Peng, Stochastic strictly contractive peaceman-rachford splitting method, 2017, arXiv preprint arXiv:1711.04955.
- [7] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, et al., Distributed optimization and statistical learning via the alternating direction method of multipliers, *Found. Trends® Mach. Learn.* 3 (1) (2011) 1–122.
- [8] G. Taylor, R. Burmeister, Z. Xu, B. Singh, A. Patel, T. Goldstein, Training neural networks without gradients: A scalable admm approach, in: *International Conference on Machine Learning*, PMLR, 2016, pp. 2722–2731.
- [9] T. Tieleman, G. Hinton, Divide the gradient by a running average of its recent magnitude. *Coursera: Neural networks for machine learning*, Tech. Rep. (2017).
- [10] J. Wang, Z. Chai, Y. Cheng, L. Zhao, Toward model parallelism for deep neural network based on gradient-free ADMM framework, in: *2020 IEEE International Conference on Data Mining, ICDM, IEEE*, 2020, pp. 591–600.
- [11] J. Wang, F. Yu, X. Chen, L. Zhao, Admm for efficient deep learning with global convergence, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 111–119.
- [12] J. Zeng, T.T.-K. Lau, S. Lin, Y. Yao, Global convergence of block coordinate descent in deep learning, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 7313–7323.
- [13] Y. Tang, Z. Kan, D. Sun, L. Qiao, J. Xiao, Z. Lai, D. Li, Admmirnn: Training rnn with stable convergence via an efficient admm approach, in: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2020, pp. 3–18.
- [14] L. Guan, Z. Yang, D. Li, X. Lu, pdADMM: An ADMM-based framework for parallel deep learning training with efficiency, *Neurocomputing* 435 (2021) 264–272.
- [15] J. Wang, H. Li, L. Zhao, Accelerated gradient-free neural network training by multi-convex alternating optimization, *Neurocomputing* 487 (2022) 130–143.
- [16] D.E. Rumelhart, G.E. Hinton, R.J. Williams, Learning representations by back-propagating errors, *Nature* 323 (6088) (1986) 533–536.
- [17] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Commun. ACM* 60 (6) (2017) 84–90.
- [18] B.T. Polyak, Some methods of speeding up the convergence of iteration methods, *USSR Comput. Math. Math. Phys.* 4 (5) (1964) 1–17.
- [19] I. Sutskever, J. Martens, G. Dahl, G. Hinton, On the importance of initialization and momentum in deep learning, in: *International Conference on Machine Learning*, PMLR, 2013, pp. 1139–1147.
- [20] J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, *J. Mach. Learn. Res.* 12 (7) (2011).
- [21] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
- [22] S.J. Reddi, S. Kale, S. Kumar, On the convergence of adam and beyond, 2019, arXiv preprint arXiv:1904.09237.
- [23] J. Zhuang, T. Tang, Y. Ding, S.C. Tatikonda, N. Dvornik, X. Papademetris, J. Duncan, Adabelief optimizer: Adapting stepsizes by the belief in observed gradients, *Adv. Neural Inf. Process. Syst.* 33 (2020) 18795–18806.
- [24] L. Luo, Y. Xiong, Y. Liu, X. Sun, Adaptive gradient methods with dynamic bound of learning rate, 2019, arXiv preprint arXiv:1902.09843.
- [25] M. Carreira-Perpinan, W. Wang, Distributed optimization of deeply nested systems, in: *Artificial Intelligence and Statistics*, PMLR, 2014, pp. 10–19.
- [26] H. Li, J. Wang, Y. Wang, Y. Cheng, L. Zhao, Community-based layerwise distributed training of graph convolutional networks, 2021, arXiv preprint arXiv:2112.09335.
- [27] X. Wen, Y. Lei, A fast ADMM framework for training deep neural networks without gradients, in: *2024 International Joint Conference on Neural Networks, IJCNN, IEEE*, 2024, pp. 1–8.
- [28] J. Xu, C. Bao, W. Xing, Convergence rates of training deep neural networks via alternating minimization methods, *Optim. Lett.* 18 (4) (2024) 909–923.
- [29] S. Liu, Z. Kong, T. Huang, Y. Du, W. Xiang, An ADMM-LSTM framework for short-term load forecasting, *Neural Netw.* 173 (2024) 106150.
- [30] L. Yang, T.K. Pong, X. Chen, Alternating direction method of multipliers for a class of nonconvex and nonsmooth problems with applications to background/foreground extraction, *SIAM J. Imaging Sci.* 10 (1) (2017) 74–110.
- [31] Y. Wang, W. Yin, J. Zeng, Global convergence of ADMM in nonconvex nonsmooth optimization, *J. Sci. Comput.* 78 (1) (2019) 29–63.
- [32] C. Chen, M. Li, X. Liu, Y. Ye, Extended ADMM and BCD for nonseparable convex minimization models with quadratic coupling terms: convergence analysis and insights, *Math. Program.* 173 (1) (2019) 37–77.
- [33] J. Wang, L. Zhao, Nonconvex generalization of admm for nonlinear equality constrained problems, 2017, arXiv preprint arXiv:1705.03412.
- [34] L.T.K. Hien, D. Papadimitriou, An inertial ADMM for a class of nonconvex composite optimization with nonlinear coupling constraints, *J. Global Optim.* (2024) 1–22.
- [35] J. Bai, M. Zhang, H. Zhang, An inexact ADMM for separable nonconvex and nonsmooth optimization, *Comput. Optim. Appl.* (2025) 1–35.
- [36] H.F. Walker, P. Ni, Anderson acceleration for fixed-point iterations, *SIAM J. Numer. Anal.* 49 (4) (2011) 1715–1735.
- [37] K. Lipnikov, D. Svyatskiy, Y. Vassilevski, Anderson acceleration for nonlinear finite volume scheme for advection-diffusion problems, *SIAM J. Sci. Comput.* 35 (2) (2013) A1120–A1136.
- [38] P.P. Pratapa, P. Suryanarayana, J.E. Pask, Anderson acceleration of the Jacobi iterative method: An efficient alternative to Krylov methods for large, sparse linear systems, *J. Comput. Phys.* 306 (2016) 43–54.
- [39] A.L. Pavlov, G.W. Ovchinnikov, D.Y. Derbyshev, D. Tsetserukou, I.V. Oseledets, AA-ICP: Iterative closest point with Anderson acceleration, in: *2018 IEEE International Conference on Robotics and Automation, ICRA, IEEE*, 2018, pp. 3407–3412.
- [40] V. Eyert, A comparative study on methods for convergence acceleration of iterative vector sequences, *J. Comput. Phys.* 124 (2) (1996) 271–285.
- [41] J. Zhang, Y. Peng, W. Ouyang, B. Deng, Accelerating ADMM for efficient simulation and optimization, *ACM Trans. Graph.* 38 (6) (2019) 1–21.
- [42] R. Rockafellar, *Convex Analysis*, Princeton U, Princeton, 1970.
- [43] Y. Nesterov, A method of solving a convex programming problem with convergence rate $\mathcal{O}(1/k^2)$, in: *Sov. Math. Dokl.*, vol. 27.
- [44] T. Goldstein, B. O'Donoghue, S. Setzer, R. Baraniuk, Fast alternating direction optimization methods, *SIAM J. Imaging Sci.* 7 (3) (2014) 1588–1623.
- [45] H.-r. Fang, Y. Saad, Two classes of multisecant methods for nonlinear acceleration, *Numer. Linear Algebra Appl.* 16 (3) (2009) 197–221.
- [46] W. Tang, P. Daoutidis, Fast and stable nonconvex constrained distributed optimization: the ELLADA algorithm, *Optim. Eng.* 23 (1) (2022) 259–301.

- [47] J. Zhang, B. O'Donoghue, S. Boyd, Globally convergent type-I Anderson acceleration for nonsmooth fixed-point iterations, *SIAM J. Optim.* 30 (4) (2020) 3170–3197.
- [48] A. Beck, M. Teboulle, A fast iterative shrinkage-thresholding algorithm for linear inverse problems, *SIAM J. Imaging Sci.* 2 (1) (2009) 183–202.
- [49] J. Wang, H. Li, L. Zhao, A convergent ADMM framework for efficient neural network training, 2021, arXiv preprint [arXiv:2112.11619](https://arxiv.org/abs/2112.11619).
- [50] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, T. Eliassi-Rad, Collective classification in network data, *AI Mag.* 29 (3) (2008) 93–93.
- [51] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, in: *Relational Representation Learning Workshop (R2L 2018)*, NeurIPS 2018, Montreal, Canada, 2018.
- [52] L. Bottou, Large-scale machine learning with stochastic gradient descent, in: *Proceedings of COMPSTAT'2010*, Springer, 2010, pp. 177–186.
- [53] S. Ruder, An overview of gradient descent optimization algorithms, 2016, arXiv preprint [arXiv:1609.04747](https://arxiv.org/abs/1609.04747).
- [54] M.D. Zeiler, Adadelta: an adaptive learning rate method, 2012, arXiv preprint [arXiv:1212.5701](https://arxiv.org/abs/1212.5701).
- [55] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (11) (1998) 2278–2324.
- [56] H. Xiao, K. Rasul, R. Vollgraf, Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms, 2017, arXiv preprint [arXiv:1708.07747](https://arxiv.org/abs/1708.07747).
- [57] A. Krizhevsky, G. Hinton, et al., Learning multiple layers of features from tiny images, 2009.
- [58] H. Attouch, J. Bolte, B.F. Svaiter, Convergence of descent methods for semi-algebraic and tame problems: proximal algorithms, forward-backward splitting, and regularized Gauss-Seidel methods, *Math. Program.* 137 (1–2) (2013) 91–129.
- [59] W. Ouyang, Y. Peng, Y. Yao, J. Zhang, B. Deng, Anderson acceleration for nonconvex ADMM based on Douglas-Rachford splitting, in: *Computer Graphics Forum*, vol. 39, Wiley Online Library, 2020, pp. 221–239, 5.



Zeinab Ebrahimi received her Master's degree in Electrical Engineering from Shiraz University, Shiraz, Iran, in 2018. She is a Ph.D. candidate in the Department of Mechanical and Manufacturing Engineering at the University of New South Wales (UNSW). Her research interests include control theory, distributed optimization algorithms in deep learning, and multi-agent systems.



Gustavo Batista received his Ph.D. in Computer Science and Computational Mathematics from the University of São Paulo, Brazil, in 2003. He joined the University of New South Wales, Sydney, as an associate professor in 2019 after working for more than ten years at the University of São Paulo, Brazil. His research focuses on time series, data stream mining, and quantification.



Mohammad Deghat received his Ph.D. degree from The Australian National University (ANU), Canberra ACT, Australia, in 2013. He was a postdoctoral researcher at the University of Melbourne, University of New South Wales (UNSW) and the Commonwealth Scientific and Industrial Research Organisation (CSIRO) in Australia. He is now a lecturer (Assistant Professor) at UNSW Sydney, NSW, Australia. His research interests include control theory and robot control, specifically in distributed multi-agent system control and nonlinear control.